Bachelor's thesis

NTNU
Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of ICT and Natural Sciences

Daniel Skjong Alnes
Aleksander Lid Nesvik

Visualization of Automatic Identification System Messages for the Norwegian Coastal Administration

Towards a safer and more secure ocean for Norway

Bachelor's thesis in Computer Science
Supervisor: Di Wu
June 2026

NTNU
Norwegian University of
Science and Technology

Daniel Skjong Alnes
Aleksander Lid Nesvik

Visualization of Automatic Identification System Messages for the Norwegian Coastal Administration

Towards a safer and more secure ocean for Norway

Bachelor's thesis in Computer Science
Supervisor: Di Wu
June 2026

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of ICT and Natural Sciences

NTNU
Norwegian University of
Science and Technology

Aleksander Nesvik, Daniel Alnes

Visualization of Automatic Identification System Messages for the Norwegian Coastal Administration

Towards a safer and more secure ocean for Norway

Bachelor's thesis in Computer Engineering
Supervisor: Di Wu
May 2026

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of ICT and Natural Sciences

NTNU

ABSTRAKT

Dette prosjektet ble gjennomført i samarbeid med Kystverket. Målet med prosjektet var å utforske og evaluere alternative visualiseringsteknikker for store datasett, samt å teste om slike visualiseringsmetoder kunne gi verdi for kunden.

Systemet ble utviklet som en applikasjon med en hjemmeside. React og TypeScript ble brukt for å lage applikasjonen, med MapLibre GL JS som basiskart og Deck.gl som datavisualiseringslag. Backend ble levert av kunden, og REST API-endepunkter ble brukt for å hente data som frontend visualiserer.

Flere visualiseringsmetoder ble implementert, inkludert klynging, varmekart, signalstyrkevisualisering og visualisering av anomaligrupper. Disse ble brukt til å evaluere hvor effektivt slike metoder kan forenkle store datasett til visuelt forståelige representasjoner.

Resultatene viser at kombinasjonen av flere ulike visualiseringsmetoder kan gi bedre innsikt for maritime operatører. Gruppen fikk positive tilbakemeldinger på den implementerte løsningen. Dette viser at prototyping med open source-verktøy og testdata kan gi verdi før full produksjon av slik funksjonalitet blir laget for interne verktøy, slik at man kan evaluere om funksjonaliteten gir verdi før produksjon.

ABSTRACT

This project was conducted in collaboration with the Norwegian Coastal Administration. The aim of the project was to explore and evaluate alternative visualization techniques for large-scale datasets, to assess whether such visualization methods would provide value to the customer.

The system was developed as a single-page application using React and TypeScript, with MapLibre GL JS as the base map and Deck.gl for data visualization layers. The backend was provided by the customer, with REST API endpoints used to request data that is visualized by the frontend.

Several visualization methods were implemented, including clustering, heat maps, signal strength visualization and anomaly group visualization. These methods were used to evaluate the effectiveness of using such visualization methods to simplify large datasets into human-readable visuals.

The results show that combining multiple different visualization methods can provide new insights for maritime operators. The group received positive feedback on the implemented solution. This demonstrates that open-source prototypes built on test data can provide value by evaluating new features before committing to full production.

PREFACE

This bachelor thesis was written as part of the final semester of the group's bachelor's degree in Computer Engineering at NTNU Ålesund. It was completed spring 2026. The project allowed the group to gain valuable new insight in open source tools, and new concepts such as web maps and data visualization. The group is grateful for the opportunity to work on a project that can bring real value to the customer, while gaining experience in new tools.

The group would like to express its sincere gratitude to its supervisor, Di Wu, for valuable guidance, feedback, and support throughout the project. Her input has been important in shaping the outcome of the thesis. The group would also like to extend its gratitude to Marius Kalvø, the main representative from the Norwegian Coastal Administration, who helped set the scope during the project and provided valuable feedback, while providing the backend for the project. Lastly, the group would like to thank the Norwegian Coastal Administration for the opportunity to work with them on this exciting project.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

AIS Automatic Identification System. 1, 2, 3, 4, 5, 15, 18, 19, 22, 27, 44, 45, 51, 56, 57, 58, 59, 60,

API Application Programming Interface. 8, 9, 17, 18, 19, 20, 23, 25, 28, 32, 34, 35, 42, 44, 51, 52,

CPU Central Processing Unit. 9,

CRUD Create, Read, Update, Delete. 9,

GeoJSON Geographic JavaScript Object Notation. 5, 13, 18, 19, 20, 22, 23, 28, 31, 32, 34, 53,

GPU Graphics Processing Unit. 9, 18, 52, 53,

HTTP Hypertext Transfer Protocol. 8, 9, 20, 31,

ID Identifier. 19, 20, 25, 35,

JSON JavaScript Object Notation. 12, 13,

MMSI Maritime Mobile Service Identity. 5, 22, 24, 25, 32, 35, 41, 43, 47, 57,

NCA Norwegian Coastal Administration. 1, 2, 3, 4, 15, 16, 18, 19, 26

NTNU Norwegian University of Science and Technology. iii, 3,

PNG Portable Network Graphics. 7,

REST Representational State Transfer. 9, 17, 18, 19, 20, 28, 32,

UI User Interface. 10, 17, 18, 20, 25,

UX User Experience. 10,

VTS Vessel Traffic Services. 1,

WebGL Web Graphics Library. 8, 9, 18, 53,

TERMS

Anomaly An AIS message flagged as anomalous, for example for an impossible speed, a position jump, or an unexpected maneuver.

Anomaly group A collection of anomalies grouped together based on shared type and geographic proximity.

Base station A shore-based station that receives AIS messages from vessels.

Basemap The underlying map layer that data visualizations are drawn on top of.

Clustering The process of grouping nearby data points based on proximity to reduce visual clutter and improve readability.

Deck.gl An open-source, WebGL-accelerated JavaScript framework for high-performance visualization of large geospatial datasets. 12, 17, 18, 19, 20, 23, 24, 35, 44

Docker An open-source platform for packaging and running applications in isolated, portable containers. 3, 13, 17, 18, 34, 44

Feature A GeoJSON object representing a single geographic entity, consisting of a geometry and a set of properties.

FeatureCollection A GeoJSON object that groups multiple Features into a single collection.

Heading The compass direction in which a vessel is pointing, given in degrees.

Heat map A visualization method that represents data density through color.

Individual anomaly A single anomaly belonging to an anomaly group, associated with one vessel.

Jamming Radio interference that disrupts the reception of AIS signals within an area.

Jumping anomaly An anomaly where a vessel's reported position jumps an impossible distance between consecutive messages.

Last activity Timestamp of the most recent message associated with an anomaly group.

Location The geographic position (longitude and latitude) of an anomaly or anomaly group.

MapLibre An open-source JavaScript library for displaying interactive, vector-tile web maps in the browser. 2, 3, 8, 17, 19, 20, 31

React An open-source JavaScript library for building user interfaces from reusable components. 2, 3, 12, 17, 18, 20, 27, 28, 30, 41

Scatter plot A visualization that plots individual data points on a coordinate plane; a scatter map plots points using longitude and latitude as the axes.

Signal strength A value representing the strength of the signal of a message between a vessel and a receiver, normalized from 0 to 1 in the dataset.

Source ID Identifier of the source associated with a message, such as the base station or satellite that received it.

Speed anomaly An anomaly where a vessel reports a speed that is physically impossible.

Spoofing The deliberate transmission of false AIS data to misrepresent a vessel's identity, position, or course.

Start time Timestamp of the first anomaly recorded in an anomaly group.

TypeScript An open-source, strongly typed superset of JavaScript that adds static type checking and compiles to plain JavaScript. 2, 3, 12, 17, 18

Unexpected maneuver An anomaly where a vessel performs a movement that is implausible for its type or situation.

Vector tiles Geographic data stored in vector form and rendered directly in the browser, used by modern web maps for smoother, customizable rendering.

Web map An interactive map displayed in a web browser.

CHAPTER

ONE

INTRODUCTION

This chapter presents the background of the project, outlines the objective and defines the requirements.

1.1 Background

Norway manages one of the world's most extensive coastlines, and the Norwegian Coastal Administration (NCA) is the national agency responsible for coastal management, maritime safety, and emergency preparedness [1]. A central part of this is monitoring of sea-faring vessels. This is done through the use of Automatic Identification System (AIS), a messaging system that transmits information about a vessel, including its heading and location. These messages are handled by "AIS Norway" which is a national tracking network operated by the NCA that combines around 90 shore base stations and a few AIS satellites [2]. Even though it was originally developed as a anti-collision tool, the AIS is now a critical part of the infrastructure for Vessel Traffic Services (VTS), search and rescue operations, and environmental protection [3].

While the NCA currently utilizes analytical tools to identify deviations in AIS data, the existing solutions lack advanced visualization capabilities such as heat maps, clustering and temporal analysis. These features are essential for operators to gain proper insight into the anomalies, because of the massive scale of data. Every day millions of AIS messages are flagged as anomalies, which makes it hard for operators to differentiate between anomalies that are technical issues and potential security threats. To solve this issue, the NCA hopes for a more intuitive, web-based prototype with new visualization methods [2].

1.2 Objective

The objective of this project is to develop a web-based program that can visualize AIS anomaly data to provide maritime experts with improved insights. The program will also serve as a tool for both live monitoring and historical analysis.

Specific goals include:

- **Advanced Map Visualization:** Implementing a map interface using MapLibre to display anomaly clusters, satellite paths, and base station locations.
- **Data Aggregation and Clustering:** Developing methods to handle high-density data without overwhelming the user interface.
- **Temporal Analysis:** Provide a time-based feature that allows users to visualize data forward and backward in time to observe development of anomalies.
- **Efficient Querying:** Building a bridge between the React frontend and the backend database to allow users to filter data by ship type, anomaly category, and time intervals.

1.3 Motivation

During a normal day the NCA gets around 80 million AIS messages. From these messages millions can potentially be flagged as anomalies.

The team chose to work on Visualization of AIS Anomaly Detection because it addresses a critical real-world problem while using modern technical approaches. Maritime security is a field of immense societal importance in Norway. The group therefore considers contributing a prototype to the NCA a very meaningful purpose for its work.

Furthermore, the group is particularly interested in the challenges of data processing and visualization as well as front-end functionality. Managing millions of data points in a web browser presents a significant technical hurdle. However, because of the group's experience in web development and data processing, this project allows the group to apply those skills to a large-scale problem.

1.4 Requirements

The requirements for the project were developed in communication with the customer. There are multiple functional and system requirements.

1.4.1 Functional Requirements

Functional requirements from the customer include support for new visualization methods such as clustering and heat maps. The system must also include filtering and anomaly representations displaying signal strength, position and anomaly type.

1.4.2 System Requirements

The solution will be implemented as a local database and a website that can query and visualize the stored data. All data processing will be handled on the device hosting the database.

- **Frontend:** Must be built using React and TypeScript
- **Map Engine:** Must utilize MapLibre or an equivalent library used by the NCA.
- **Data Querying:** The application must be able to query a database to handle the large amounts of data.

1.5 Software and Technology Stack

The following technologies were used throughout the development of the project:

- **React & TypeScript:** Frontend framework and language
- **MapLibre GL JS:** Mapping engine for high-performance maps.
- **Customer provided backend** A Docker container-based backend system built in Go, used for data storage and querying.
- **NCA Test Datasets:** Simulated AIS anomaly data.

1.5.1 Stakeholders

- **The Norwegian Coastal Administration (Kystverket):** The primary stakeholder and ”client.” They provide the problem description, the tech stack requirements, and the AIS data.
- **The Project Team:** Two NTNU students responsible for the design, implementation, and testing of the visualization prototype.
- **Maritime Operators:** The end-users who require an intuitive interface to maintain maritime safety.

1.6 Thesis structure

- **Chapter 2 – Theory:** This chapter presents the theoretical background of the project. This will give the underlying theory on how mapping visualization works, how previous research in visualization of in browser map systems, and how clustering and visualization of data is achieved.

- **Chapter 3 – Methods:** This chapter describes methodology and hardware the group used during the project, what libraries and programming langauges that were used, also how the project was structured and developed from start to finish, why explaining why certain tools and methodology were used.

- **Chapter 4 – Results:** This chapter describes the results the group accomplished, how it was accomplished, while describing the solution in detail.

- **Chapter 5 – Discussion:** this chapter discusses how the project went in detail. It explains the group’s reflections on the project, how the plan was followed, if there were discrepancies between the plan and the actual progression of the project.

- **Chapter 6 – Conclusion:** This chapter serves as a conclusion to the report and discusses future development on the software that can be done.
- **Chapter 7 – Societal Impact:** This chapter describes any societal or environmental consequences the product has

CHAPTER

TWO

THEORY

This chapter gives the theoretical background for the bachelor project, explaining technologies and principles used.

2.1 Automatic Identification System (AIS)

The Automatic Identification System (AIS) is a system used to transfer information about a ship, including its location and heading, to base stations, satellites and other ships, on coasts around the world. The system is prone to errors due to incorrect configurations, area jamming, or message spoofing. The NCA processes this data to detect errors in AIS messages. In practice, this pipeline identifies millions of anomalous messages per day, as illustrated in Figure 2.1.1.

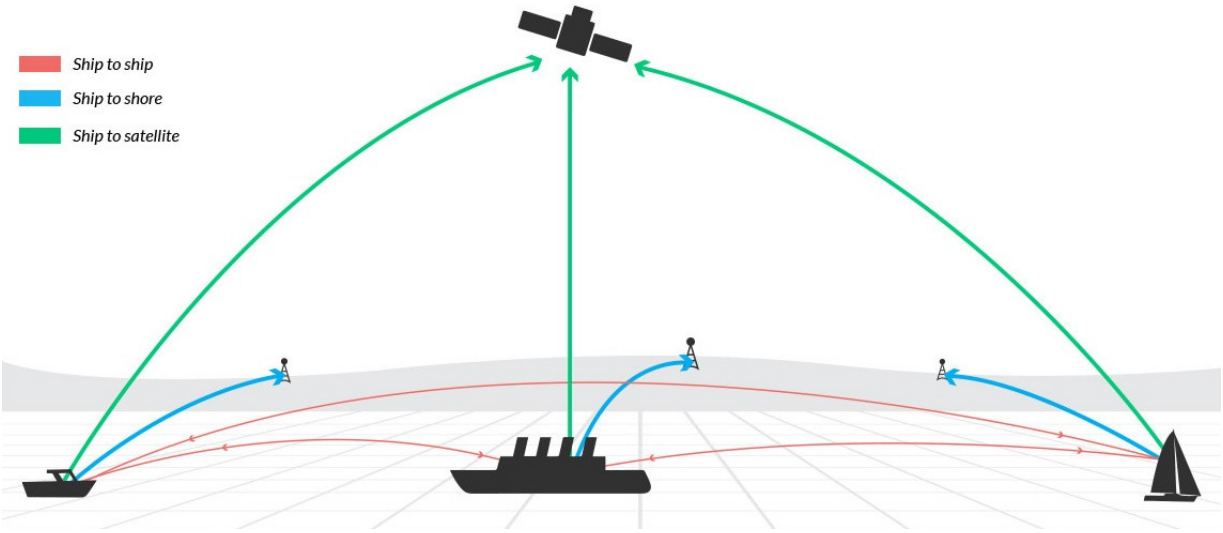


Figure 2.1.1: Example of AIS message flow [4].

2.2 Digital Twin

A digital twin is a digital representation of a physical object or system. An AIS digital twin primarily represents a ship's location and speed, but may also include additional

information such as the vessel type and heading. The main purpose of a digital twin is to enable real-time tracking and to provide historical data for further analysis [5].

2.3 Anomalies and Anomaly Groups

Anomalies refer to detected irregularities from AIS messages. Anomalous messages are of differing types, including speed, unexpected maneuver, and jumping anomalies, where these pertain to if a ship's message shows impossible speed or distance covered. Anomalies also contain a Maritime Mobile Service Identity (MMSI), being a ship's unique identification number. Anomaly groups are collections of anomalies of similar type. These groupings are created through proximity between anomalies. Anomaly groups themselves are in the GeoJSON data format as Features within a FeatureCollection. See Section 2.15.4.

2.4 Signal Strength

Signal strength measures the received power of an electric signal over distance, measured in decibels. Weak signal strength leads to loss of data and clarity, with strong signals mitigating these factors. Signals travel through the air and other materials, distance affects the strength of the received signal, with long distances requiring more time to reach its recipient [6]. For AIS, signal strength can be used to observe errors in anomaly messages. AIS messages should in theory transmit their message to the nearest base station or satellite. By reading the signal strength of a message, it can be flagged as inconsistent with the expected value, indicating an error or anomaly in the data.

2.5 Data Visualization

Data visualization is the practice of displaying information in a more understandable format than raw data. As datasets grow in magnitude, it becomes increasingly complex to parse information from them, thus data visualization tries to mitigate this problem by displaying the data in new forms. In the context of the project this encompasses different visualization methods such as heatmaps and clusters, which give a simplified view of where data is concentrated [7].

2.6 Human-Centric Data Visualization

Human visual perception excels at spotting patterns and extracting meaningful information. However, there are limits on the amount of information humans can parse. Therefore, visualization techniques focus on presenting data in a way that better fits humans pattern recognition. This is accomplished in various ways, one of which being reducing visual clutter through removing or aggregating data. [8]

2.6.1 Heat-Map Visualization

Heatmaps are used for data visualization, to show relationship between variables, such as the distribution of data points per type, the quantity of data is represented through color, where higher concentrations have a darker color than less concentrated areas of points. This leads to intuitive visualization of datasets, as shown in Figure 2.6.1 [9].

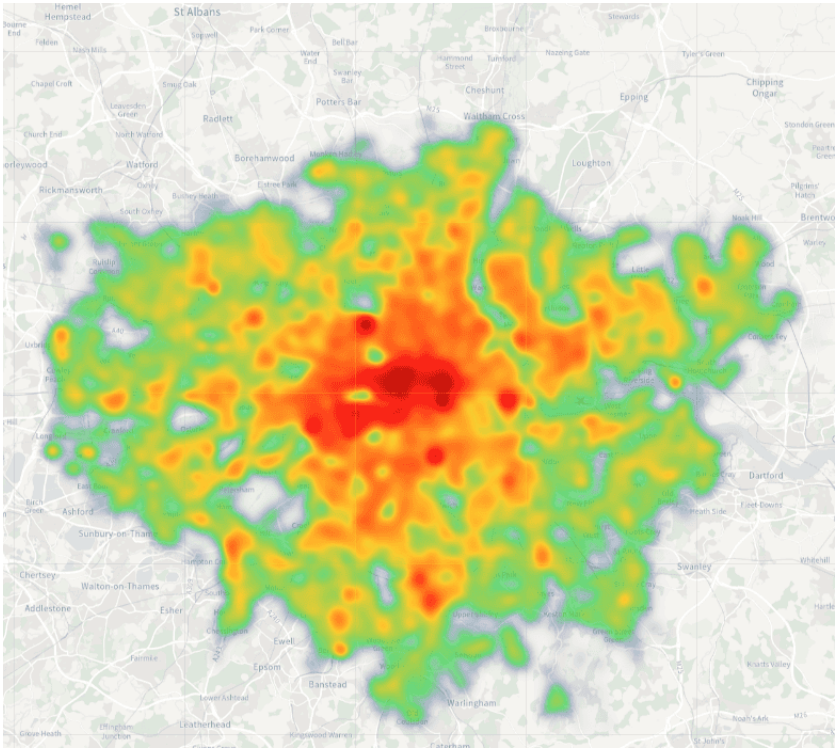


Figure 2.6.1: Example of a heat map visualization [10]

2.7 Clustering

Clustering pertains to a method of aggregating data points into groups or clusters. Clusters can be created based on various factors such as movie genre, user details or anomaly type. There are multiple different clustering algorithms that serve different purposes, such as density or proximity-based methods, as illustrated in Figure 2.7.1 [11].

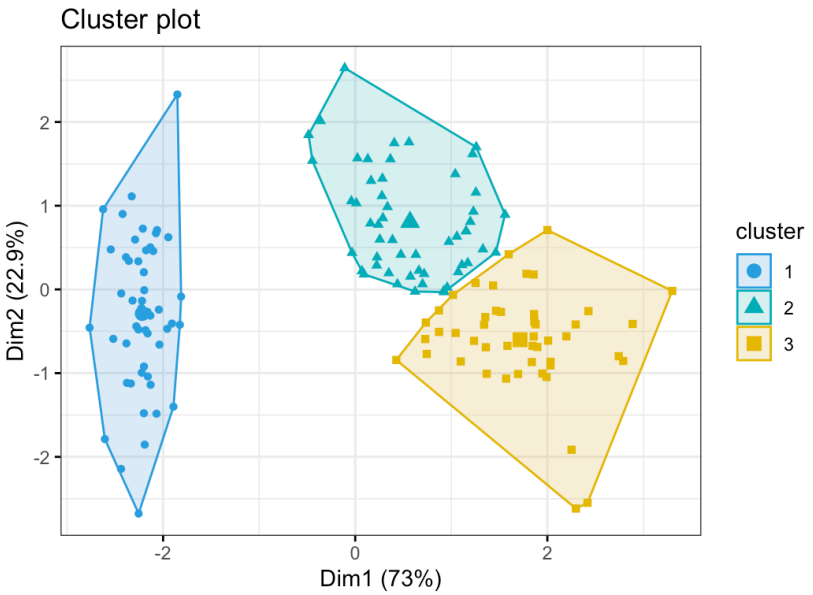


Figure 2.7.1: Example of clustered data points using the k-means clustering method [12].

2.7.1 Scatter Map (Geographic Scatter□Plot)

A scatter plot is a visualization method used to show correlation between variables or points, where each point is plotted on a x,y coordinate plane. Scatter plots are used in visualizing relationships between points of data, allowing for patterns and outliers to appear in data visually. A scatter map is an approach to scatter plots where longitude and latitude replace x and y coordinates. Resulting in points being visualized on a map. This allows the visualization of geographic data, giving insight on patterns and congregation of data points [13].

2.8 Web-Mapping Stack

Web maps are used for geographic visualization on the internet. Modern web maps allow users to interact with maps on websites in an intuitive way, making geographic data easier to explore and understand. Web maps are created by having a basemap which is the first layer showing the map itself, without additional data layers. On top of the basemap, additional layers can be added, that are shown above it. These layers can be, data points of incidents happening in a geographic area, charting of train tracks in a country, and other spatial data depending on the application.[14]

2.8.1 Vector Tiles

Vector tiles are what is currently used by most web map providers. Vector tiles store geographic data in vector format to show the contents of the map itself, instead of PNG images that were used in the past. Vector tiles allow for more styling and customization than PNG image tiles, and have led to smoother interaction with web maps, at the cost of requiring more processing power [15], as illustrated in Figure 2.8.1.

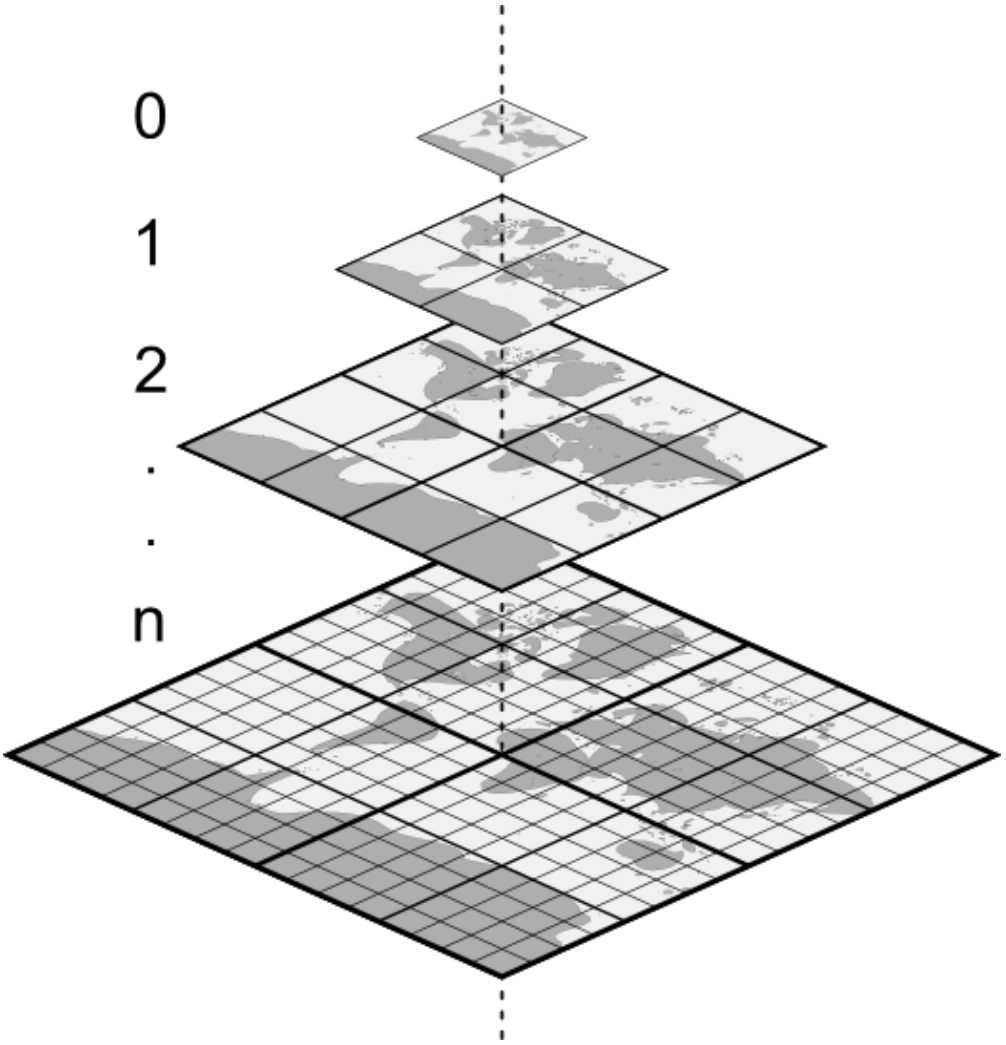


Figure 2.8.1: Illustration of vector tile zoom levels [16].

2.8.2 MapLibre

MapLibre is an open source web mapping library, built around vector tile rendering. It started as a fork of mapbox-gl-js. It utilizes vector tile rendering with Web Graphics Library (WebGL) for efficient map rendering and interaction. MapLibre is used for rendering geospatial data into human-readable visualizations [17, 18].

2.9 API

An Application Programming Interface (API) is a method that allows different software to communicate with each other, most commonly through the client-server architecture. APIs define the protocols and rules for how software communicates, what format data is sent in and what data is exchanged [19, 20].

2.9.1 Client-Server Architecture

The client-server architecture is structured around server computers, processing requests from clients over a network. This communication is done through protocols like Hypertext Transfer Protocol (HTTP), allowing the server to respond appropriately to client requests [21].

2.9.2 REST Architecture

Representational State Transfer (REST) is a software architecture designed around constraints concerning the client-server model. REST is stateless, meaning the entire context of the communication is stored on the client, and the client sends the entire information related to the request to the server. This means that the server does not maintain state, while additionally REST is designed around a uniform interface, denoting that data transfer is optimized for the standard web communication, rather than to individual applications [22].

2.9.3 REST API

A REST API (Representational State Transfer Application Programming Interface) is an API designed around the constraints of the REST architecture. REST APIs are designed around creating, reading, updating, and deleting data (CRUD). These operations are done through an HTTP request, with an example being GET used as a request to access data on a server. REST APIs are commonly used for web APIs, chosen for their scalability and handling of different data formats [23].

2.10 Hardware Acceleration

Hardware acceleration is a method of increasing performance in tasks predominantly handled by the Central Processing Unit (CPU). As defined by ScienceDirect Topics [24], this is achieved by offloading highly parallelizable computing tasks from the general-purpose CPU to other hardware in the computer such as the Graphics Processing Unit (GPU). Because the GPU has significantly more parallel processing, this frees up the CPU to perform other tasks, while providing a massive performance increase on heavy visual rendering tasks that use hardware acceleration [24].

2.10.1 WebGL

WebGL is a graphics API that is used in both 2d and 3d graphics rendering in compatible web browsers. WebGL allows for hardware acceleration, leading to more efficient rendering performance. WebGL is supported by most modern browsers, in addition to not requiring browser plugins, leading to ease of use. WebGL is highly used in fields like data visualization applications and general graphics rendering in the browser [25, 26].

2.11 Software Engineering Principles

Software systems are frequently based on established software engineering principles and methodologies developed by pioneers in the field. These principles are followed to improve code quality and software design.

2.11.1 Cohesion and Coupling

Coupling and cohesion are foundational principles in structured software design [27]. Coupling refers to how tightly different parts of a software system depend on each other. If parts of the software have high coupling, it creates a ”ripple effect” where it becomes harder to make changes to the project without other parts of the project breaking [27]. Cohesion describes whether parts of the software are closely related and designed to complete a singular task, rather than being unfocused and serving multiple unrelated purposes [27]. The goal for coupling and cohesion is to have high cohesion and low coupling, which in turn leads to better-designed, maintainable software [27].

2.11.2 Separation of Concerns

Separation of concerns is a software design principle, aimed at separating system functionality into cohesive units that address specific concerns. This leads to independent components of the system with specific responsibilities, resulting in a more modular design [28].

2.11.3 Single Responsibility Principle

The single responsibility principle dictates that a part of software should only have one responsibility, such that code is modular and reusable. As a result of following this principle, code becomes easier to extend and maintain [29].

2.11.4 Documentation

Code documentation is the process of explaining through comments how the parts, methods and flow of a project functions. Documentation is crucial for clarity and maintainability of code, whereas undocumented code can cause confusion and unexplained complexity leading to less effective development [30].

2.11.5 Refactoring

Refactoring is an essential part of maintaining and creating high-quality code. Refactoring is the restructuring of existing code, where it is changed to better follow software design principles, while maintaining the same functionality. As a result of this, code becomes easier to maintain and extend, improving overall code quality [31].

2.12 Design

Design is a central component of software development, defining the appearance and functionality of the final product. In the context of data visualization, design also affects the insights gained from the visualized data and how effectively an analyst can work with the system [32, 33].

2.12.1 Wireframes

Wireframes are a tool used to create design blueprints for applications and websites. Wireframes are normally employed during the early phases of a development project. Wireframes allow for sketching UI/UX to varying degrees of detail, which the final software design will be based on [34, 35].

2.13 Testing

Testing of software allows developers to locate errors in code and design. It helps ensure that the system behaves as intended.

2.13.1 Manual Testing

Manual testing is the process wherein the developer tests the software through various methods, including evaluating the end user experience or through direct interaction with the application or software. This allows bugs to be identified and user experience to be tested in a single process. Manual Testing is a flexible approach to where defects outside of automatic test scope can be identified and fixed [36].

2.14 Project Management

Project management refers to the process of coordinating and structuring the development of a project. There are different project management methodologies, that

aim to improve coordination, management, and overall effectiveness of the project.

2.14.1 Agile Development

Agile Development relates to software development approaches emphasizing incremental development with early feedback. This creates a loop of development and feedback that ensures projects avoid unnecessary, unguided work on redundant systems or features. Agile methods are frameworks that give guidance on how to develop software, following the agile principles, with Scrum being the most common one [37].

2.14.2 Scrum Framework

Scrum is a project management framework designed after agile principles. Scrum emphasizes iterative development by segregating work into an iterable time-period called a sprint cycle. Following the Scrum framework requires creating specific roles in the team, including the Scrum Master, who coaches and monitors the Scrum process. The product owners represent the stakeholders whom the product is created for and who are responsible for providing feedback during development. The development team is the last role, consisting of a small amount of members, responsible for the implementation of the project.

The work in a Scrum project is represented through Scrum artifacts, divided into three distinct parts. The product backlog is a list of features and requirements defined by the product owner. The sprint backlog contains features, bugs and implementations being worked on for a team's current sprint cycle, that is intended to be complete by the end of a cycle, while an increment is an end goal of a sprint.

Sprints are executed as time-boxed iterations where the development team works on selected backlog items. A sprint itself is planned beforehand, defining the goals to be achieved. After a sprint is completed, a sprint review and retrospective is held. The review is an informal showcase of the work done, while the retrospective evaluates the sprint itself, reflecting on strengths and weaknesses the sprint had. Sprints are typically conducted in a weekly or biweekly cycle, with frequent feedback from the product owner.

A system that is utilized by developers is a daily Scrum or as a more general term a daily stand-up. This is a daily meeting where developers discuss current work, and any issues they have encountered. The amount of time spent on a daily stand-up is around 15 minutes. The next day of work is then planned based on the outcome of the stand-up. Daily stand-ups facilitate communication between team members, with different areas of expertise, and allow for flexible changes to project planning [38, 39].

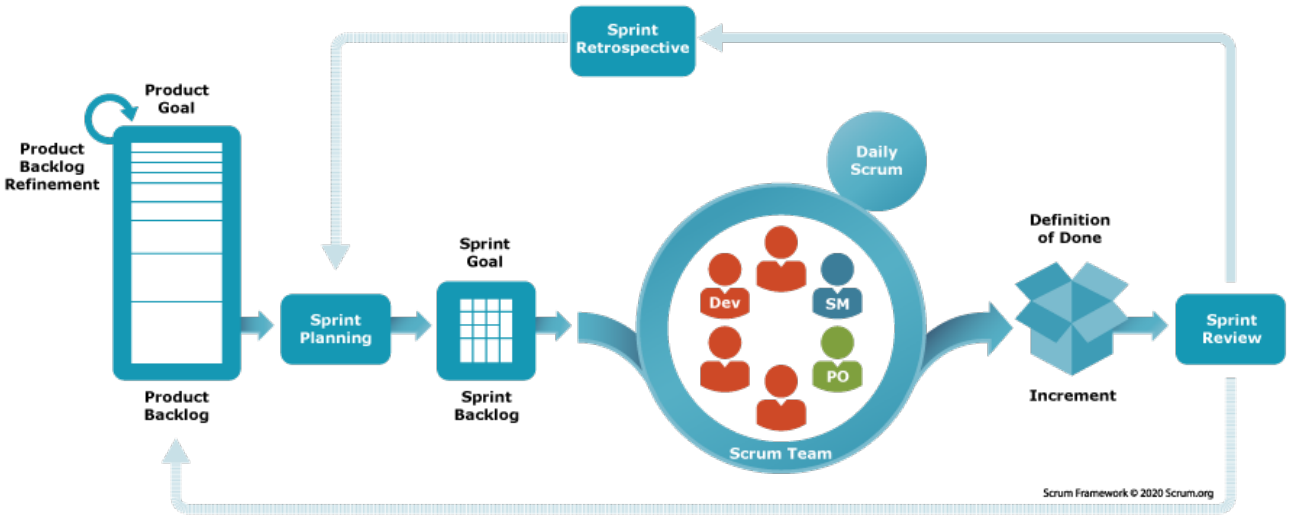


Figure 2.14.1: Illustration of scrum lifecycle[40].

2.15 System Architecture

System architecture describes the structure and behavior of a system. It defines how the components within a system are organized and how those components interact with each other to deliver the intended functionality of the application.

2.15.1 React

React is a JavaScript library for building user interfaces through a component based architecture [41]. Following the official ”Thinking in React” methodology, components are reusable pieces of code that receive data through ”props” and combine to define the user interface to be rendered [41]. React’s approach provides scalability, modularity and reusability to projects that implement it. React is commonly paired with TypeScript, TypeScript is a strongly typed superset of JavaScript [42]. TypeScript adds static types to Javascript, enabling strongly typed React applications, catching errors at compile time, and improving overall type safety [42].

2.15.2 Deck.gl

Deck.gl is an open source, high-performance, WebGL-based visualization framework originally created by Uber. Visualization is handled through layering, where each layer can contain its own visualization method. Layers allow for event handling, and custom rendering of complex visualization methods, based on datasets. Datasets containing geographic data are frequently used with map providers to map data points, to geographic coordinates, enabling human-readable visualization of complex data [43, 44, 45, 46].

2.15.3 JSON (JavaScript Object Notation)

JavaScript Object Notation (JSON) is a data structure created for ease of readability for humans, being based on JavaScript. JSON data is architecturally designed around objects with name/value pairs, that can represent data as objects or arrays.

The JSON format is widely supported and can be parsed across most programming languages [47].

2.15.4 GeoJSON

GeoJSON is a data structure based on JSON, that represents geographic data. GeoJSON encodes the specifics of a geographic feature,which can contain data on specific points, objects or areas. Geographic features are stored as objects called Features in a FeatureCollection [48].

2.15.5 Supercluster.js

Supercluster.js is a JavaScript library, created for clustering GeoJSON data features based on proximity. Clustering is calculated through the zoom level of the map instance, on the points in the browser window currently visible. Clustering is done through the hierarchical greedy clustering method, grouping points based on radius around various points chosen from the dataset. The high performance of the library is achieved by reusing previously computed data, which increases performance in zoom and drag operations [49, 50].

2.15.6 Docker

Docker is a platform designed to separate the infrastructure used to build software from the application itself. This allows the software to be run in isolated instances. These instances are called docker containers. Containers are standardized packages of the software, containing all dependencies required for the application. As a result of this the docker container can run the application without needing any other dependencies, creating a system where applications can be run on different operating systems and hardware, without requiring downloads of software dependencies or programming languages[51, 52].

CHAPTER

THREE

METHODS

This chapter outlines the development process, tools, frameworks, system architecture, and testing strategies employed throughout the project.

3.1 Project Management and Process

This section describes how the project was organized, including the development methodology, team structure, meeting routines, communication, and task management tools used throughout the project.

3.1.1 Scrum Framework

The Scrum framework, described in Section 2.14.2 and illustrated in Figure 2.14.1, was employed to structure the development process. It was introduced by the supervising lecturer, who provided pre-filled sprint templates and documentation to support its use in group-based projects. Scrum was chosen for its suitability with iterative development and its continuous feedback loop, which allows for early changes in direction. This reduced the time investment of adopting the framework, while ensuring that sprint artefacts such as the backlog and retrospectives were consistently maintained throughout the project.

Scrum was also well suitable due to the project's biweekly meetings with both the supervisor and the customer at the NCA. Each sprint provided opportunities to present and discuss ongoing development, allowing scope adjustments to be made based on feedback. This helped the group particularly when dealing with technical constraints, such as limitations or issues with the AIS dataset or uncertainty around feasibility or scope for specific visualization features. The iterative structure allowed focus on the backlog until discussions about scope or features with the customer at the NCA rather than following a fixed plan. The project's internal sprint structure was organized into weekly sprints, which followed issues created on GitHub related to each week's iteration. At the end of each sprint, a short retrospective meeting was conducted to evaluate the progress of the group, planning what to focus on the next sprint. This informed prioritization of backlog items, if any issues were not completed during the sprint. This was used when planning the following sprint.

3.1.2 Daily Stand-Ups

Daily stand-up meetings were established as an integral part of the sprint workflow, in accordance with the Scrum practices described in Section 2.14.2. The primary purpose of the stand-ups was to maintain accountability between team members and to further facilitate collaborative problem-solving between team members when problems blocking further development occurred.

In a two-person team, the risk of one member being stuck on a problem without the other member being aware is likely. Daily meetings allowed such issues to be discussed early, preventing them from blocking progress. They were also utilized as a daily forced break for the team. Sustained work without regular breaks does not equate to consistent productivity. In addition, they served as a way to quickly discuss a problem for further collaborative work after the stand-up meeting. While the daily stand-up was followed once daily, there were multiple days where meetings were conducted while walking, combining discussion of the project with a short break.

3.1.3 Team Composition and Roles

The project was conducted by a two-member team, with responsibilities divided to ensure that core Scrum artifacts were maintained consistently throughout the project. One member served as a Scrum Master, taking control of sprint documentation, backlog maintenance, and handling of sprint meetings and stand-ups. The other member was responsible for handling meeting notes. All remaining responsibilities, including development, testing and design, were shared equally.

3.1.4 Meeting Structure

Biweekly meetings were held with the supervisor to review academic progress and discuss the project's technical direction. Customer meetings with the NCA were arranged biweekly and focused on presenting completed work and gathering feedback while clarifying the scope of the project. The outcomes from both meetings were recorded in meeting notes and were stored in a dedicated Discord document channel, ensuring that decisions remained accessible to both team members throughout the project and that the project scope remained stable. Occasionally, biweekly meetings were rescheduled in agreement with the supervisor or customer due to scheduling conflicts. These conflicts were resolved through communication with the relevant party. No formal meeting invitation documents were used, as scheduling was handled directly through email or Slack communication, with meeting notes being written for each scheduled meeting.

3.1.5 Communication

The main communication platform the team utilized was Discord. Discord was selected due to its support of persistent text channels and voice communication. The platform was used for planning meetings, task coordination and general communication. It primarily served as a channel for communication and storing information. Outlook was also utilized to contact the supervisor and plan meetings. The customer at the NCA used Slack to communicate, primarily to plan meetings, ask for progress updates, and answer technical questions.

3.1.6 Task Management

Task tracking was handled using GitHub Projects, which provided a shared Kanban-style board across the team's three repositories: one for the frontend, one for the backend, and one for administrative documents such as meeting notes and sprint

documentation. The tool was selected as the task management tool, as the team was already using GitHub for version control. Issues were created and tracked through workflow stages to ensure workload transparency and motivation. These workflow stages were: To Do, In Progress, In Review, and Done. These stages ensured that team members could easily see work progress through a sprint and pick up extra tasks in the backlog when tasks were completed without requiring communication with the other team member. Combined with the three-repository structure, this ensured issues were separated based on the repository it was on, while keeping issues at their relevant repository.

GitHub branching was used to isolate development between developers. Each member worked on dedicated branches on tasks assigned to that member. Upon completion, branches were reviewed by the responsible member and merged to the main branch.

3.1.7 Tools and Platforms

3.1.7.1 Development Tools

- **Visual Studio Code:** Primary development environment used by all team members and was selected for its extensive extension system and integrated terminal.
- **Git & GitHub:** Used for version control, branching, pull requests, and task management via GitHub Projects. GitHub was adopted due to the groups prior knowledge with the tool, while task management and sprint being handled through it due to recommendation from the supervisor.
- **Docker:** The backend was containerised using Docker to ensure a consistent and reproducible runtime environment. This was preferred by the customer due to ease of use of Docker containers.
- **Swagger:** Used for API documentation and testing of backend endpoints, originally set up by the customer. This was selected by the customer for comprehensive API testing and documentation.

3.1.7.2 Design Tools

- **Figma:** Used for designing wireframes and UI mockups for the web application. This was used due to its support for collaboration between members, while being a free tool that came recommended by the customer.

3.2 System Architecture

The architecture follows a client-server model (Section 2.9.1). The backend is implemented in Go and running inside a Docker container exposing a REST API (Section 2.9.3). Swagger is used for API documentation, and the frontend is built with React and TypeScript retrieving the data it needs through API calls to the backend.

3.2.1 Architecture Overview

The system is based on a client-server architecture consisting of a React frontend, Docker backend, and database. The React frontend communicates with the backend through REST API calls using Axios, which results in the backend sending a dataset based on request parameters. This dataset is then visualized through Deck.gl in the frontend, layered over a MapLibre map.

3.2.2 Technology Stack

The technology stack was primarily determined by the customer's existing infrastructure. Since the prototype is intended to work with the NCA's systems, using their technology choices ensures compatibility. It also allows the team to leverage the customer's expertise if questions arise. Tools for which the customer has no preference, such as the map provider and visualization libraries, were decided by the group based on research and project requirements.

3.2.2.1 Frontend Technologies

- **JavaScript:** Programming language used by React. This was chosen due to customer requirement, and that it allows for comprehensive interaction implementation to the project.
- **React:** React was used as a JavaScript library, due to its reusable component-based architecture and props.
- **TypeScript:** TypeScript was used with React to implement more type safety to the project.
- **MapLibre GL JS:** Mapping engine used for rendering interactive, high-performance maps with WebGL acceleration. This was chosen due to being recommended by the customer, and being open-source.
- **Deck.gl:** This is the library used for visualization of anomaly data, and was chosen due to being widely used in the industry, being GPU-accelerated and designed to handle large-scale datasets. In addition to providing a large suite of visualization methods.
- **Axios:** Library used for REST API communication. This was used due to simplifying data retrieval from the backend API.
- **React-window:** Library used for virtualization of lists. This was chosen due to simplicity of implementation.
- **SuperCluster:** Library used for calculation of clusters based on anomaly data, used for visualizing clusters. This was chosen due to using hierarchical greedy clustering (Section 2.15.5), and being designed to calculate on GeoJSON datasets.
- **chroma-js:** Library used for signal-strength color interpolation, generating the red–yellow–green color gradient applied to the anomaly group and individual anomaly layers.
- **lodash:** Utility library used for debouncing the sidebar search input, avoiding filtering on every keystroke.
- **lucide-react:** Icon library providing the UI icons used throughout the interface.

3.2.2.2 Backend Technologies

- **PostgreSQL:** The database storing the AIS anomaly test data, supplied by the customer.
- **Swagger:** Swagger generates API documentation through annotations in the Go backend. A Swagger UI endpoint is available locally for exploring and testing the API.
- **Docker:** Used for containerizing the backend services.

The backend was originally provided by the NCA as a working codebase, pre-populated with test data for development, since actual AIS anomaly data is confidential. The customer supplied a Go-based starter backend, distinct from their primary .NET/C# stack. The team forked this repository and updated it throughout the project. The most notable addition was a master query endpoint described in Section 3.4. The test dataset consists of three anomaly types: speed, maneuver, and jumping anomalies. Anomalies are grouped by type and proximity, ranging from thousands to a single anomaly entry per group.

3.2.3 Anomaly Group And Anomalies

As mentioned in Section 2.3, anomalies and anomaly groups are represented as GeoJSON FeatureCollections. Anomaly groups and anomalies are implemented as features in a FeatureCollection, containing properties and geometry. Geometry defines the geographic location of an anomaly, with properties containing specific data such as anomaly type, of which there are three types: unexpected maneuver, speed and jumping anomalies. This includes other data such as the ID of the anomaly, its heading or last activity date.

3.2.3.1 MapLibre

MapLibre is used as the base map of the application. As an interactive map is crucial for the project, it was decided to use MapLibre based on its strengths, such as being an open source project that provides all required mapping functionality, while having integrated Deck.gl support for adding layers on top through an overlay. The utilization of vector tiles in MapLibre is another factor in why it was chosen, where vector tiles lead to improved performance and interactivity, while having a smoother picture than alternatives (see Section 2.8.1 and Figure 2.8.1).

MapLibre serves as the foundational map layer for the application. While MapLibre itself can be used to visualize, Deck.gl was chosen for visualizing the dataset, whereas MapLibre is focused on handling zoom, interaction and the base map, while visualization layers are rendered on top of MapLibre.

3.2.4 Deck.gl Layer Integration

Deck.gl is the visualization library used as a layer on top of MapLibre. The decision to use the library came from the requirement to visualize hundreds of thousands of points concurrently on the map. Performance is one of the main factors driving the choice, as Deck.gl is created specifically for the task of visualizing large datasets.

Deck.gl is responsible for rendering anomaly data as visualization layers above the base map. The layers are defined through functions that generate layer objects based on the dataset received from the backend.

3.3 Frontend

This section describes the frontend architecture, the design process that shaped it, and the communication patterns between frontend and backend. The resulting implementation is presented in Section 4.3.

3.3.1 Frontend Architecture

The frontend architecture follows a client-driven data flow where data is retrieved from the backend through REST API calls using Axios (Section 2.9.3). The received

data is stored in React state as a GeoJSON FeatureCollection, which is passed to individual Deck.gl layers responsible for rendering the visualizations. This is done after a map instance is created through a map container class using MapLibre. Deck.gl is subsequently used for rendering data points overlaid on the map. Interactivity is handled through React components, where state changes control filtering and updating of the rendered layers. Individual Deck.gl layers also implement click event handlers, which update application state and trigger the rendering of additional information components or different visualization layers (Section 2.15.2).

3.3.2 Frontend component structure

The frontend is implemented as a single-page application, centered around the main map. The interface is structured as a set of components, with the map acting as the central component. Different components are positioned over the map allowing for filtering, anomaly search and context sensitive information displays for anomalies.

The application maintains shared state of the currently loaded dataset, which is passed to components that require it. Active filters are maintained through state management, where any change in the current filter is observed by other components, triggering visualization changes.

The application uses no routing, as it is a single page application, with all components dynamically displayed and updated based on application state.

3.3.2.1 REST Communication with Axios

REST communication between frontend and backend was handled using Axios. This was chosen to enable asynchronous HTTP requests, without blocking the user interface. This enabled dynamic updates to the dataset on request, such as querying for specific anomaly types or date range. Axios allowed the application to update in real-time, where an API call returns a promise of the data requested. This allowed for the UI to update after data has been retrieved. This is intended to facilitate a more responsive user interface for end users [53].

3.3.2.2 GeoJSON Data Handling

GeoJSON serves as the container for all geographic data for anomalies in the project. This file format was chosen in discussion with the customer, based on GeoJSON's strengths such as human readability and simplicity in use and parsing (Section 2.15.4).

The frontend retrieves and interprets FeatureCollection data sent from the backend, for rendering purposes. Each Feature represents an anomaly group or anomaly, with metadata containing information about type and geographic location.

The design of the application processes the anomaly groups themselves, while allowing for individual anomalies from a group to be rendered in its own layer. This is done through first retrieving an individual anomaly group through an API request. The returned group provides access to its ID, which identifies the associated anomalies. An API request is sent related to these anomalies, wherein a new layer is created visualizing them [48].

3.3.3 Design Process

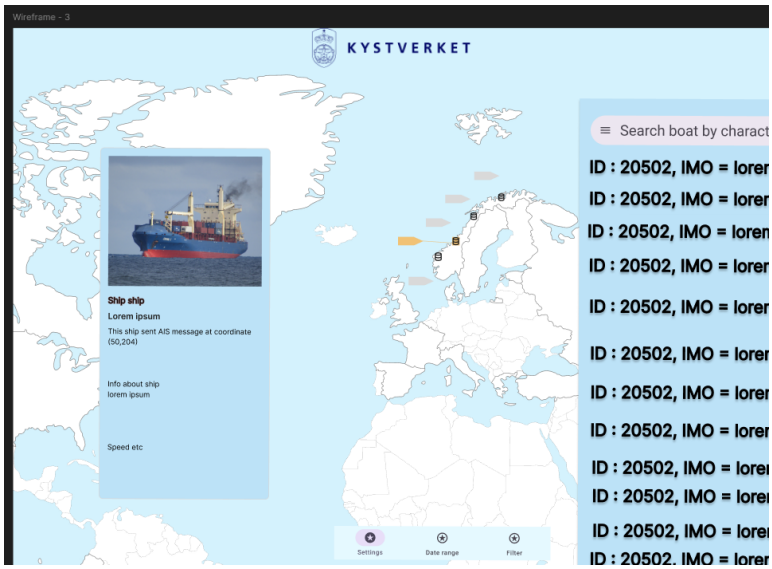
The design process was guided by the design and wireframing theories described in Section 2.12 and Section 2.12.1. The process started early with customer meetings where the customer's existing system was presented, and other systems that are similar. This was then used as a design basis for the group's implementation of the project.

3.3.3.1 Design Approach

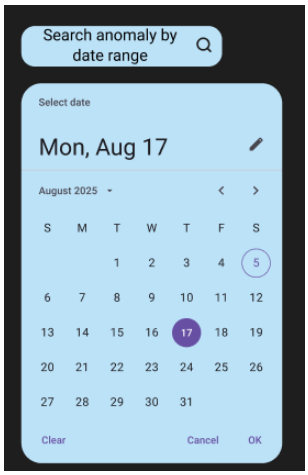
The design of the user interface followed a three-stage approach. First, existing visualization systems within geographic and maritime sectors were reviewed to identify common interaction patterns and layout conventions used in this domain. Second, the design principles introduced in Section 2.12 were applied to the interface. This ensured the system followed principles like simplicity and consistency, helping reduce the risk of operators becoming overwhelmed working with large amounts of data. Third, the project was adapted to fit the specific project requirements. The design of the application changed throughout the project due to customer feedback, although the main UI design followed the wireframes created at the start of the project.

3.3.4 Wireframes

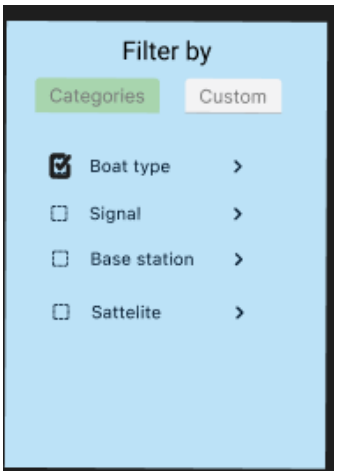
The wireframes, described in Section 2.12.1 and shown in Figure 3.3.1, were created in Figma at the start of the project. The application was decided to be based on a single page design, containing a user interface to allow for interaction and visualization. The design was to contain the map, as the core part of the application, with it taking up most of the page. On top of the map, other components like a search bar and visualized anomalies and base stations were to be overlaid, with filtering through various variables like date range were to be implemented in a popup on the page. The design was also to have an info component for anomalies.



(a) Main page wireframe



(b) Date picker component



(c) Filter component

Figure 3.3.1: Wireframes created in Figma showing main UI layout and components

3.4 Backend and Data Flow

This section describes the backend infrastructure and the data flow between it and the frontend, building on the architecture in Section 3.2.

The backend infrastructure and test database were provided by the customer, pre-populated with test data since actual AIS anomaly data is confidential. The team's main contribution to the backend was a master query endpoint. This endpoint accepts all filter parameters (anomaly type, MMSI, and time range) and returns only the matching data. Instead of getting all the anomaly groups and filtering on the frontend, all variables get sent to the backend which queries the data again. This approach was chosen because keeping old data and filtering client-side was slow and inefficient.

The anomaly set is saved as a FeatureCollection in GeoJSON, with a FeatureCollection being the entire set of geographic objects. The anomaly data is built up such that all anomaly groups are part of a collection.

When a request is made, the backend first retrieves the relevant anomaly groups, based on the frontend request, returning a GeoJSON FeatureCollection.

3.5 Visualization Implementation

The visualization methods implemented in this project were selected based on requirements from the customer and refined throughout the development process. This section describes the set of visualization techniques used in the system and how they were implemented.

3.5.1 Clustering

Clustering was implemented at the request of the customer as a way to reduce visual clutter and improve human readability when visualizing large datasets. It follows the general clustering theory described in Section 2.7 (Figure 2.7.1). As mentioned in Section 2.15.5, Supercluster.js is used for calculating clusters based on zoom level and the current map bounds. Supercluster calculates clusters based on the base dataset acquired from the API. The clustering behavior is controlled through a fixed radius parameter of 40 pixels, where anomalies within this pixel radius are aggregated and shown as a single point on the map. Recalculation happens on zoom and drag operations, where clusters split into smaller parts as points move outside the radius. Map interactions such as zooming and dragging result in clusters being retrieved for the current zoom level and map bounds. Supercluster itself was chosen for its ease of use with GeoJSON, while being designed for fast calculations, through the use of hierarchical greedy clustering, while reusing previous calculations (Section 2.15.5).

The clustering data itself is used with Deck.gl (Section 2.15.2). Visualization through Deck.gl is accomplished through creating a custom layer, consisting of a label-layer, used to display the amount of anomalies within each cluster, placed above a scatterplot layer (Section 2.7.1). This combines to create the clustering layer. This layer is re-rendered whenever the clustering output changes due to updates in zoom level or map bounds. Additionally, cluster visualization includes dynamic sizing based on amount of anomalies inside a cluster, leading to clusters being scaled in size proportional to their anomaly count.

3.5.2 Heat Map

A heat map layer was implemented for visualization of the full dataset (Section 2.6.1, Figure 2.6.1). Heat map visualization is done through the Deck.gl HeatMapLayer. Each data point contributes equally to the weighted visualization, where the sum of points within a fixed 40-pixel radius is used to calculate the color shown [45], correlating to density of points.

The visual representation of density is accomplished through densities being given weight, which then is translated to a color scale. Lower densities correlate to a yellow color while higher densities show a redder color.

The heatmap updates when the zoom level or view changes, leading to recalculation of density, as points are further apart on zoom-in.

3.5.3 Base Station Layer

A base station layer was required to support visualization of signal strength anomalies. The layer was implemented through an icon-layer with Deck.gl, with an icon

of a database being used to symbolize the base station on the map. This layer was then to be used together with other layers related to it, showing how anomalies at sea and base stations communicate.

3.5.4 Anomaly Group Layer

The anomaly group layer is a visualization layer that gives a detailed representation of a single anomaly group (Section 2.3). The anomaly group visualization layer is created through a custom Deck.gl layer. This layer consists of a signal strength layer, using data from the individual anomalies inside the group, that are connected to a satellite or base station with a signal strength number value. This value is used to color from red through yellow to green, indicating whether the signal is weak (red) or strong (green).

3.5.5 Individual Anomaly Layer

An individual anomaly visualization layer was requested by the customer during the later stages of development. This layer was implemented as a scatter plot, with each point representing a singular ship, from the anomaly group. The individual points are colored from red through yellow to green based on signal strength, with green considered a strong signal and red a weak one. The signal strength value is normalized from 0 to 1 in the database, and this normalized value determines the point's color. The visualization layers use the following Deck.gl layer types: ScatterplotLayer for individual anomaly and cluster points, HeatmapLayer for the heat map, IconLayer for base stations and satellites, LineLayer for signal strength connection lines, and TextLayer for cluster count labels.

3.5.6 Scope and Feature Prioritization

The project scope was decided through ongoing discussions with the customer. Hexagonal binning was discussed with the customer as a possible feature during a meeting, but it was not part of the written project brief. Due to time constraints it was not implemented in the final prototype. The scope of the project changed throughout development, leading to some new features being requested, some of which were cut. The team prioritized clustering, heat maps, and the filter system based on customer feedback since these provided the most immediate analytical value.

3.6 Filter and Interaction

The filtering and interaction system builds on the visualization layer described in Section 3.5. This section will explain the implemented interaction and filtering systems.

3.6.1 Filter System

The filter system was scoped early in the project. Group-level filtering was chosen over per-anomaly filtering, because filtering per-anomaly would include more data fields such as source ID, signal strength and more which was concluded to be too computationally expensive for interactive use on a large database. Since the main scope of the project was focused on anomaly groups, this scope made sure that the filter would be focused on performance while aligning with the visualization goals of the project. For this, four filterable fields were selected: MMSI, anomaly type, start date, and end date. These were chosen to support the operators' workflows since they were the information most likely to be investigated in practice. Things such as

isolating anomalies by time period, type, or MMSI. Then using that filtered data to find specific areas of elevated anomaly density.

3.6.2 Temporal Filtering

A simple date-time range selector was selected for temporal filtering. The design goal was to let operators restrict the visualization to a chosen time period and then use other tools to further identify special factors of points in that period. The filter was constrained to remain within the scope of the project while providing simplicity and feasibility of implementation.

3.6.3 Satellite Implementation

Satellite connections were implemented as a layer that would only be visualized when a point is clicked to follow the project scope and make sure that complexity stayed low. This decision made sure that operators could not clutter the screen with satellites and allows them to see the points and connections easily.

3.6.4 Interactive Features

The filtering controls are designed for simple, interactive operation. A native HTML date-time input was implemented for date selection, and a drop-down listing the available types was implemented for anomaly type selection. A third component was for inputting numbers which was implemented for MMSI selection. These standard interfaces were chosen to keep the filtering workflow familiar to other systems while also allowing operators without prior training to use the prototype.

3.7 Testing

Testing of the system was employed through manual evaluation of the application in accordance with the testing theory described in Section 2.13 and Section 2.13.1.

3.7.1 Backend API Testing

Testing of the backend was limited due to the fact that it was primarily created by the customer, meaning that formal tests were not created by the group. However manual testing of API endpoints was performed through a Swagger UI, to verify the validity of requests, while also being tested in the application itself.

Additional testing was performed through logging of retrieved data on the browser console, allowing correctness to be verified by inspecting the returned responses.

The Swagger UI was also used for endpoint testing while the backend was running. This was done by providing different parameters such as an anomaly ID, and observing the corresponding API responses.

3.7.2 Frontend Testing

Frontend testing was performed manually through direct interaction with the application. This included testing the core functionality.

The components were tested to ensure that correct information was displayed, this was verified through console logging and observing the received data, while also using the Swagger API to observe whether the data returned from the endpoint matched what was shown in the components.

Testing of specific methods in the code was performed through console logging to identify errors and verify if the correct methods were being called. In addition, all

filtering and search functionality was tested directly in the application, ensuring correctness of data.

3.7.3 Performance Observation Methodology

Performance was evaluated through manual observation of the application during use. No automated performance testing was implemented. The system was tested using anomaly datasets ranging from 10,000 anomalies to over 100,000 anomaly groups. The focus of the performance testing was to ensure that the application remained responsive for end users and that rendering behavior did not exhibit major performance issues.

To ensure performance, certain optimisations were implemented, such as triggering cluster recalculation on the map's move event so that it fires once per completed pan or zoom gesture rather than continuously during the operation. Utilizing virtualization in the sidebar so that the full dataset was not rendered in the list at all times, and debouncing the sidebar search input by 2000 milliseconds to avoid filtering on every letter.

3.8 User Feedback

User feedback was gathered primarily through the biweekly customer meetings as described in Section 3.1.4. These were used to gain feedback on design, primarily based on the customer's perspective, in order to reduce complexity and adjust scope while ensuring that the product was developed according to their requirements.

The feedback was used to iteratively guide development throughout the project. Features were both scrapped and added as requests based on ongoing communication. This helped minimize wasted work on unwanted features, while allowing changes to be incorporated early in the development process.

At the end of the project, a video of the application, along with questions to employees at the NCA, was sent to the customer representative. This allowed the group to receive feedback from actual operators currently using the NCA's internal system, in order to assess whether the implemented functionality would provide value to them.

CHAPTER

FOUR

RESULTS

This chapter outlines the results of the project, explaining the functionality and design of the completed project.

This chapter presents the resulting application, a locally hosted visualization tool for AIS anomalies and anomaly groups. The system implements clustering, heat map, base station, anomaly group, and individual anomaly visualization layers, supported by a filtering system and interactive features. The technology stack was partly guided by the customer, with React and the map provider specified, while remaining tool choices were made by the group. The following sections describe the delivered system, organized in the same order as the Method chapter: project management, system architecture, frontend, backend and data handling, visualization, filtering and interaction, testing and performance, customer feedback, and project outcome.

4.1 Project Management

The project followed the Scrum-based process described in Section 3.1.1. Each sprint included a sprint planning session, daily synchronization meetings, and a retrospective. Bi-weekly meetings were held with both the supervisor and the customer at Kystverket, though scheduling occasionally extended the interval to three weeks. At the end of the project, the GitHub Projects contained 106 completed issues split across the three repositories: 67 on the administrative and thesis repository, 36 on the frontend, and 4 on the backend. The administrative repository in particular contained many smaller documentation and process tasks tracked as individual issues.

Team roles were maintained throughout the project. Beyond the Scrum Master and meeting-notes responsibilities, the remaining work was divided according to each member's preference and the tedium of the task, ensuring that both members always had work to focus on and that the workload was evenly distributed. Exceptions were made in cases of illness, where role-specific work was usually delayed until the responsible member was able to handle it.

4.2 System Architecture

The final system architecture, resulting from the design described in Section 3.2, consists of a React frontend communicating with a backend REST API. The frontend is responsible for rendering visualization layers and handling user interaction, while the backend provides GeoJSON-formatted anomaly data and database generation. The architecture separates visualization, filtering, and data retrieval into different parts. The following diagrams illustrate the system's structure, data flow and user interaction.

4.2.1 Use Case Diagram

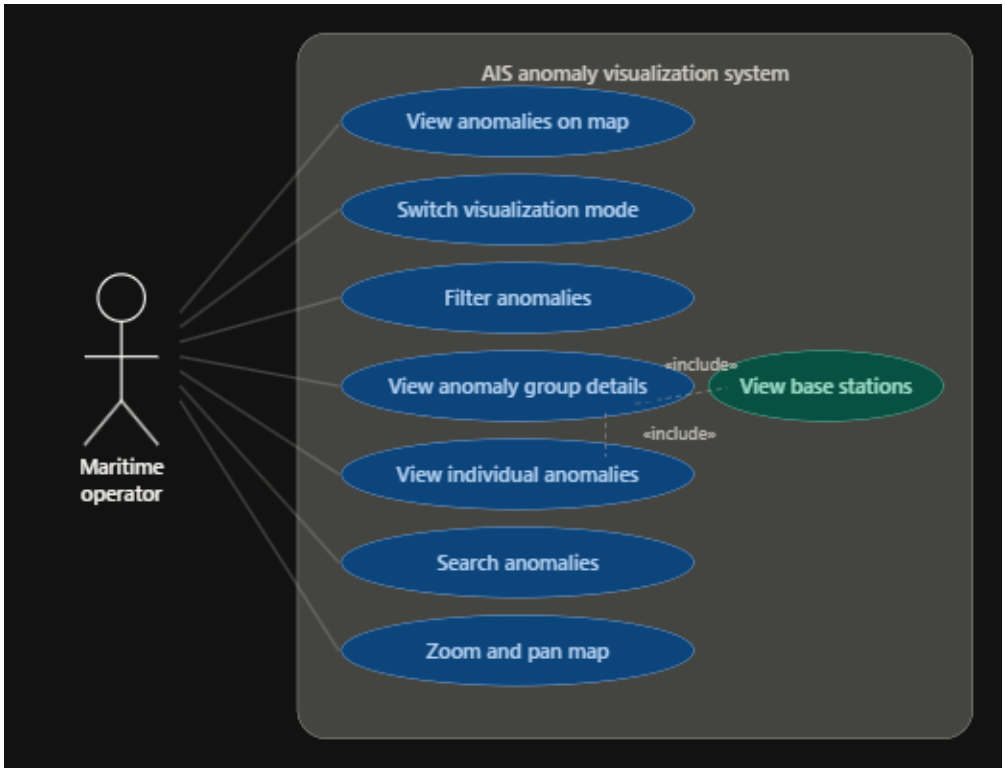


Figure 4.2.1: Use case diagram showing the expected interactions between a maritime operator and the system's core features

The use case diagram in Figure 4.2.1 illustrates the expected user experience for a maritime operator. It showcases the core capabilities delivered by the system, allowing an operator to view anomalies, switch between different visualization layers, and access specific information regarding individual anomalies and anomaly groups.

4.2.2 Package Diagram

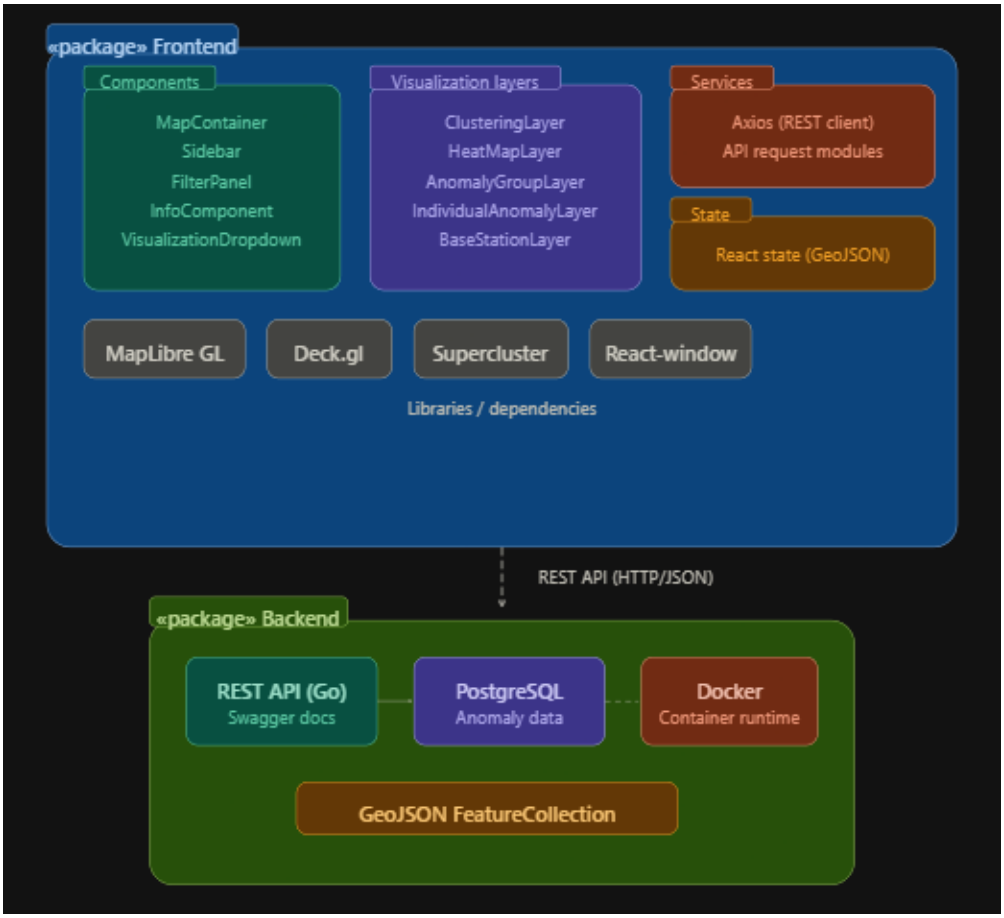


Figure 4.2.2: Package diagram showing the modular directory structure of the frontend repository

The package diagram in Figure 4.2.2 outlines the modular directory structure used in the frontend repository. Files are put into folders related to their function, such as components being isolated within a dedicated component folder, with necessary subfolders. This aligns with standard React component patterns as described in Section 2.15.1.

4.2.3 Sequence Diagram (Data Flow)

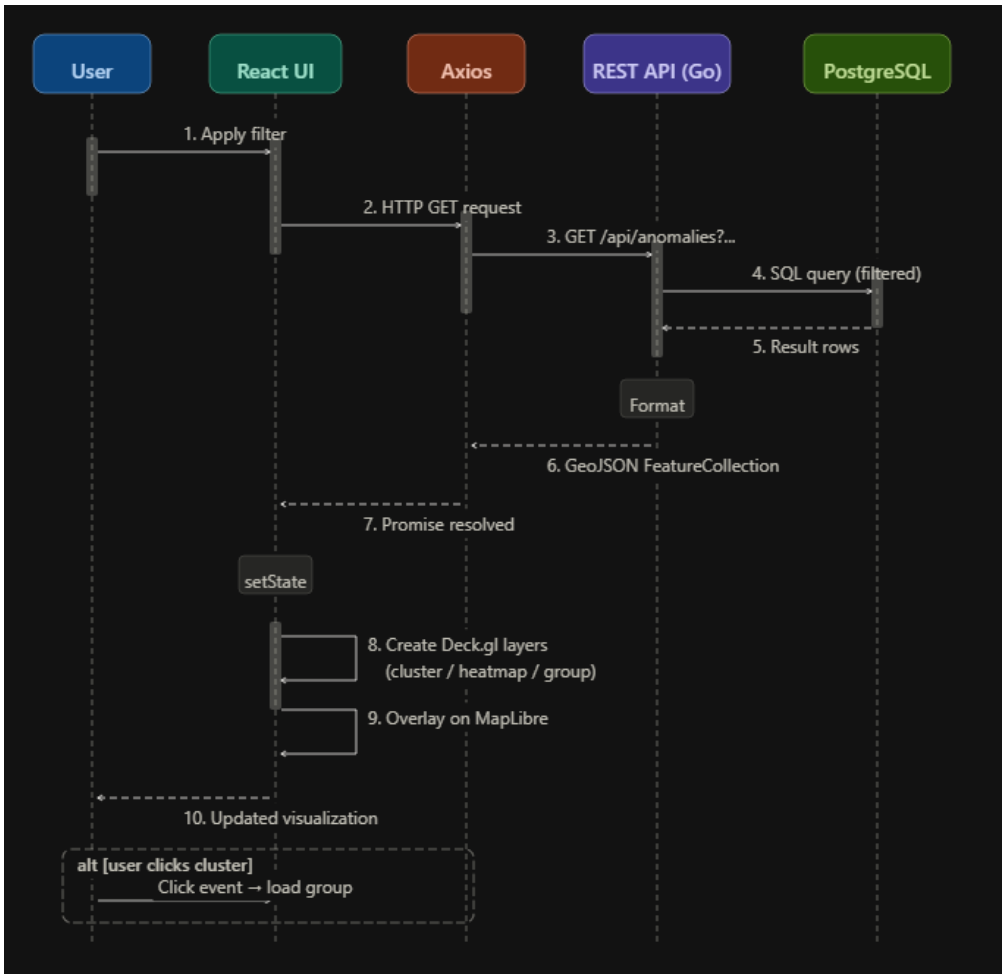


Figure 4.2.3: Sequence diagram illustrating the data flow from a user filter action to an updated visualization layer

The sequence diagram in Figure 4.2.3 maps the data flow from user interaction to visual state change. As illustrated in the image, if a user requests to apply a filter, an HTTP GET request is sent to an endpoint, which then queries the database. This then returns a GeoJSON FeatureCollection as a promise, and is then asynchronously accessed through React. This then creates the Deck.gl layer requested, and overlays it on the map, leading to an updated visualization for the end user.

4.2.4 MapLibre

MapLibre initializes itself upon application boot, in a bright style used for readability for users. After its initialization, it is responsible for interaction, in dragging and zooming the map. Interaction leads to states being updated that other parts of the program can use, such as clusters being updated through zooming, and map location. Visualization functionality is something MapLibre has, being able to natively visualize scatterplots and other layers.

4.3 Frontend

The resulting frontend of the project was created following the method in Section 3.3.3. Figure 4.3.1 shows the finished frontend.

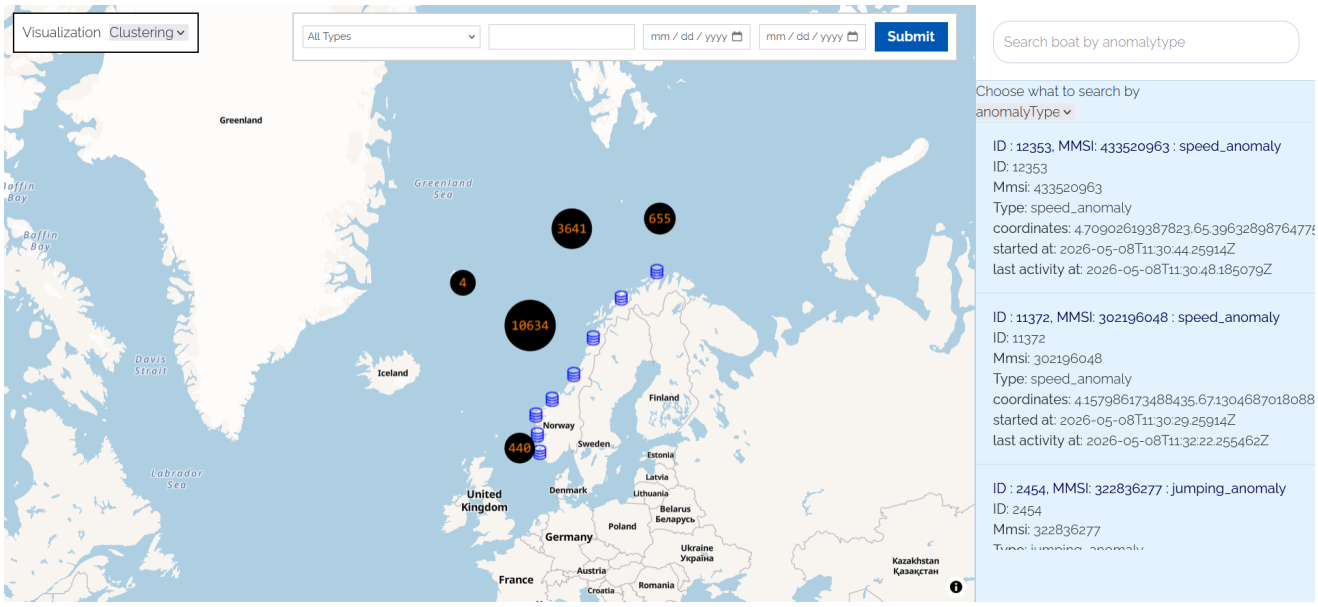


Figure 4.3.1: The completed single-page application showing the map view with visualization overlay, sidebar, and filter component

A singular page was created for the application, being the only page needed for all functionality. As shown in Figure 4.3.1, the user interface is made up of multiple parts. The base map takes up the center of the application, with visualization overlaid on top of the map.

Components such as a sidebar on the top right contain the entire dataset, allowing for searching of anomalies based on type or MMSI. A visualization dropdown menu allows switching between the two primary visualization modes — clustering and heat map. The anomaly group and individual anomaly views are entered by clicking a map point, forming a drill-down navigation rather than a separate top-level selection. On the top middle of the page is a filtering component, allowing users to change the displayed dataset by filtering on anomaly type, date range, and MMSI. Compared to the wireframes created during the design process (Section 3.3.3), the final layout diverged in several ways based on customer feedback and the features added during development. The filter component was moved to the top of the page and simplified: where the wireframe filter contained multiple custom-value fields and expandable lists, the delivered filter is a single wide bar containing only anomaly type, MMSI, and a start and end date, reducing clutter on the main page. After several visualization methods were added, a permanent area was placed in the top left for switching between the clustering and heat map visualizations. The sidebar stayed close to its wireframe but now spans the full height of the right side of the page. The anomaly info card also remained close to the original design, with the addition of a button that displays the anomalies within a group, which was requested by the customer.

4.4 Backend and Data Handling

The backend, described in Section 3.4, was developed by the customer and provided as part of the project infrastructure. It exposes a REST API used by the frontend to retrieve anomaly data. Anomaly data is stored as a GeoJSON FeatureCollection, with each feature representing a single anomaly group.

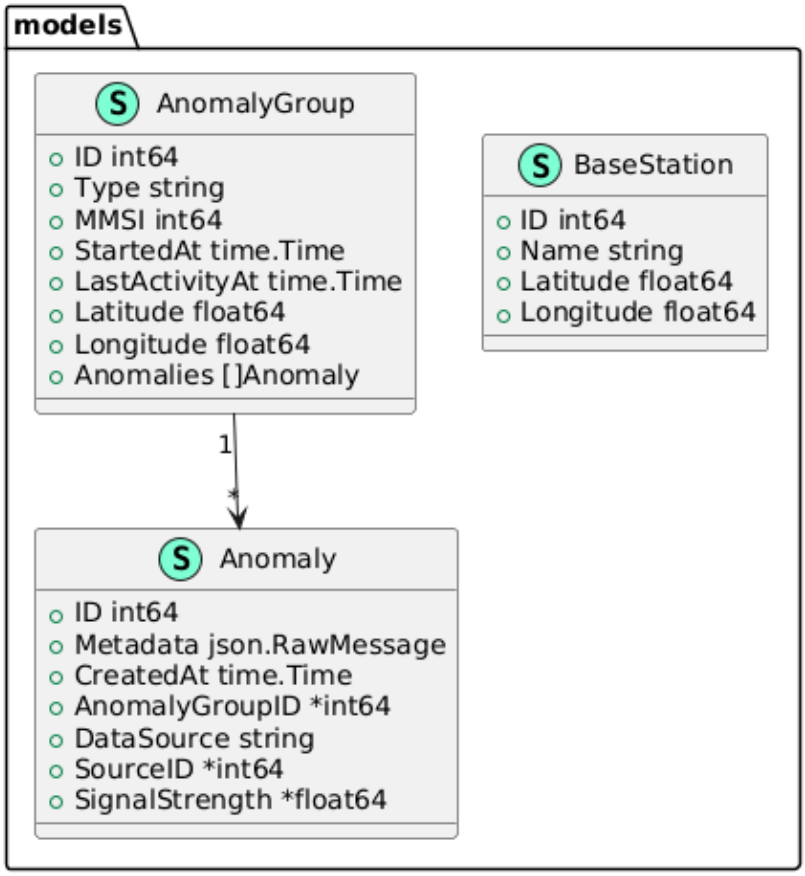


Figure 4.4.1: Database model diagram showing the relationship between anomaly groups and individual anomalies

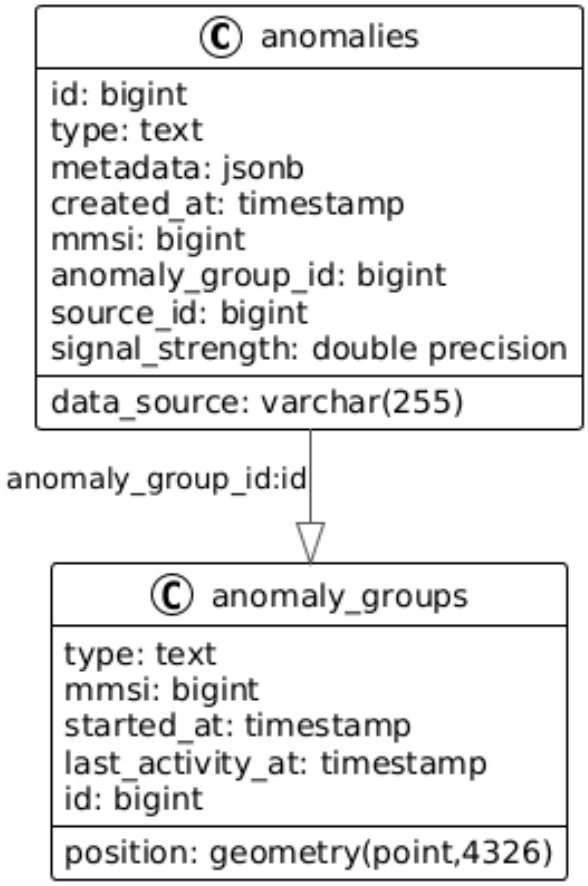


Figure 4.4.2: Database schema showing table structure, field definitions, and data types

Figures 4.4.1 and 4.4.2 show the backend database structure. An anomaly group contains anomalies, where the anomaly group itself has the start time and last activity time, sourced from the first instance of an anomaly being created in a group. Anomalies themselves have signal strength and their own data, with the group containing general data related to the group such as the general area of the anomalies, represented as location.

The anomalies and anomaly groups are generated by the Docker backend, with the amount of anomalies set through a configuration file. Specific commands to generate anomalies and build the database are used, leading to a database filled with an amount of points set by the developer. The Docker backend running allows for the frontend to send requests to API endpoints to request data, while the frontend itself runs through localhost.

4.4.1 GeoJSON Data Structure

The final GeoJSON structure is shown in Figure 4.4.3.

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [
          4.70902619387823,
          65.39632898764775
        ]
      },
      "properties": {
        "id": 12353,
        "lastActivityAt": "2026-05-08T11:30:48.185079Z",
        "mmsi": 433520963,
        "startedAt": "2026-05-08T11:30:44.25914Z",
        "type": "speed_anomaly"
      }
    },
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [
          4.157986173488435,
          67.13046870180882
        ]
      },
```


Figure 4.4.3: Example GeoJSON FeatureCollection returned by the backend API, with anomaly groups as individual features

Anomaly data is completely integrated in a FeatureCollection, with the individual features being anomaly groups. These features are individually processed in the frontend, as a dataset, through Deck.gl.

Each Feature carries the properties defined in the Terms list and are ID, MMSI, anomaly type, signal strength, heading, last activity, and location, which the frontend reads to drive the visualization layers.

4.4.2 Backend Query and Response Behavior

Figure 4.4.4 showcases an API endpoint, created as a master query allowing for filtered requests on all relevant variables. The master query accepts four filter parameters: anomaly type, MMSI, and a start and end date. It returns the matching anomaly groups, where each group contains its individual anomalies, and each anomaly in turn includes its base-station links and signal strength values. This endpoint was the group's main contribution to the backend infrastructure provided by the customer, replacing frontend-side filtering of the full dataset with a single backend query (Section 3.4).

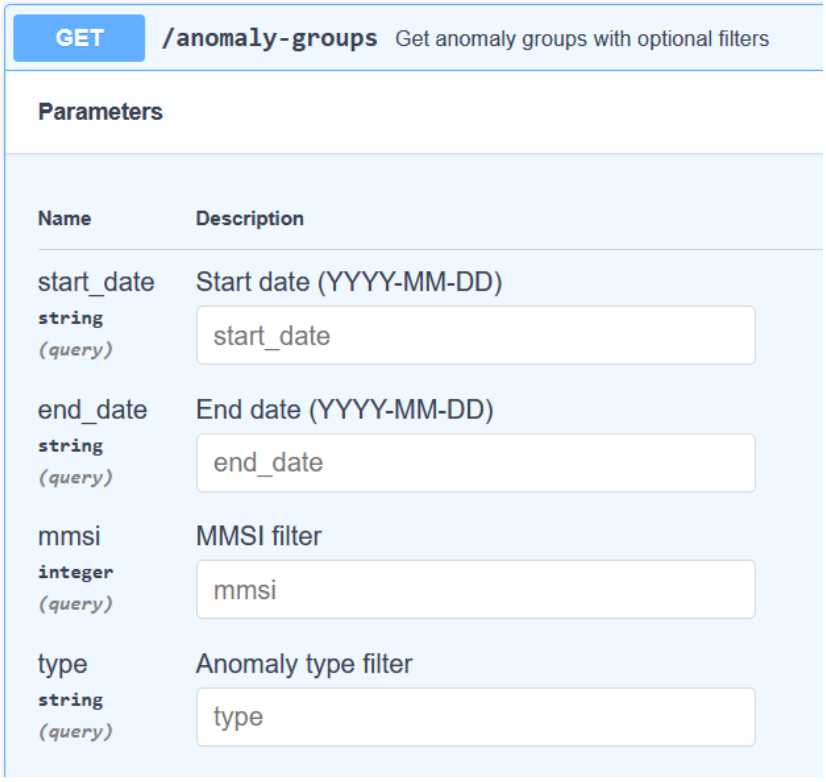


Figure 4.4.4: Swagger UI showing the master query endpoint with all available filter parameters

General endpoints for retrieving all anomaly data, as well as filtered datasets, are implemented. Requesting filtered data leads to the base dataset changing to contain the new data, while in other cases such as the sidebar having its own data variable to not change the main map data through search.

4.5 Visualization Results

All visualization methods described in the Method chapter were implemented in the final system. The system includes clustering, heat map, base station, anomaly group, and individual anomaly visualization. Additional proposed features were not implemented.

4.5.1 Clustering Visualization

The clustering visualization was implemented as described in Section 3.5.1. The clustering approach uses Supercluster.js to calculate clusters through proximity based on radius and zoom level (Section 2.15.5). This approach produces grouped anomaly points, with numeric labels indicating the number of anomalies in each cluster, as shown in Figure 4.5.1.

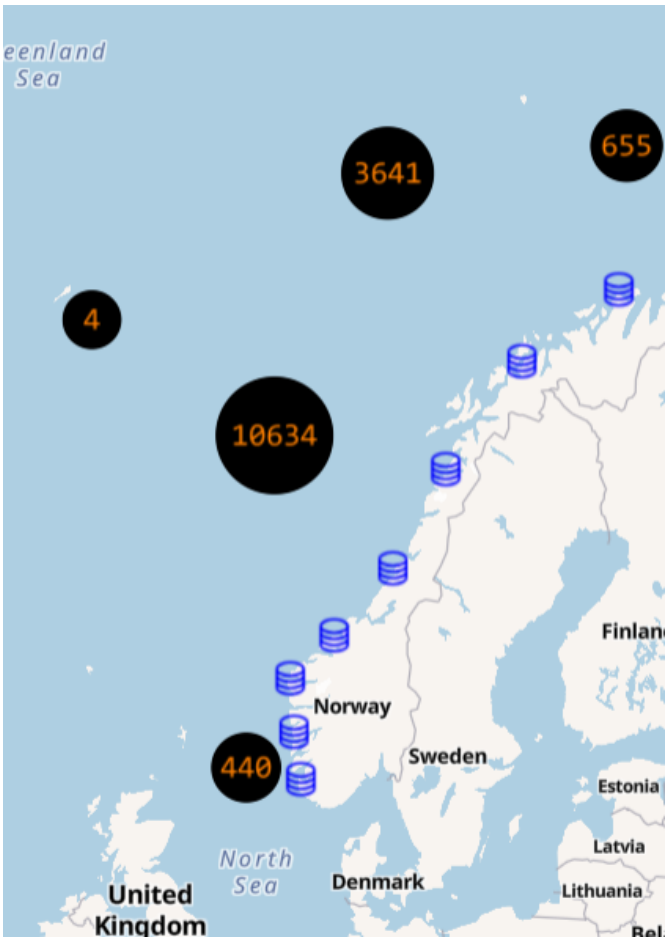


Figure 4.5.1: Clustering visualization showing anomaly groups aggregated by proximity, with numeric labels indicating the group count per cluster

At the zoomed-out level shown in Figure 4.5.1, the full dataset aggregates into a small number of clusters positioned over the Norwegian Sea off the coast of Norway. The largest cluster holds 10,634 anomalies, with neighbouring clusters of 3,641, 655, 440, and 4 anomalies; zooming in splits these into progressively smaller clusters. A higher density of points in a cluster leads to that cluster being shown as larger on the map.

4.5.2 Heat Map Visualization

The system includes a heat map visualization layer. The heat map is implemented as in Section 3.5.2. An example of the implemented heat map visualization is shown in Figure 4.5.2.

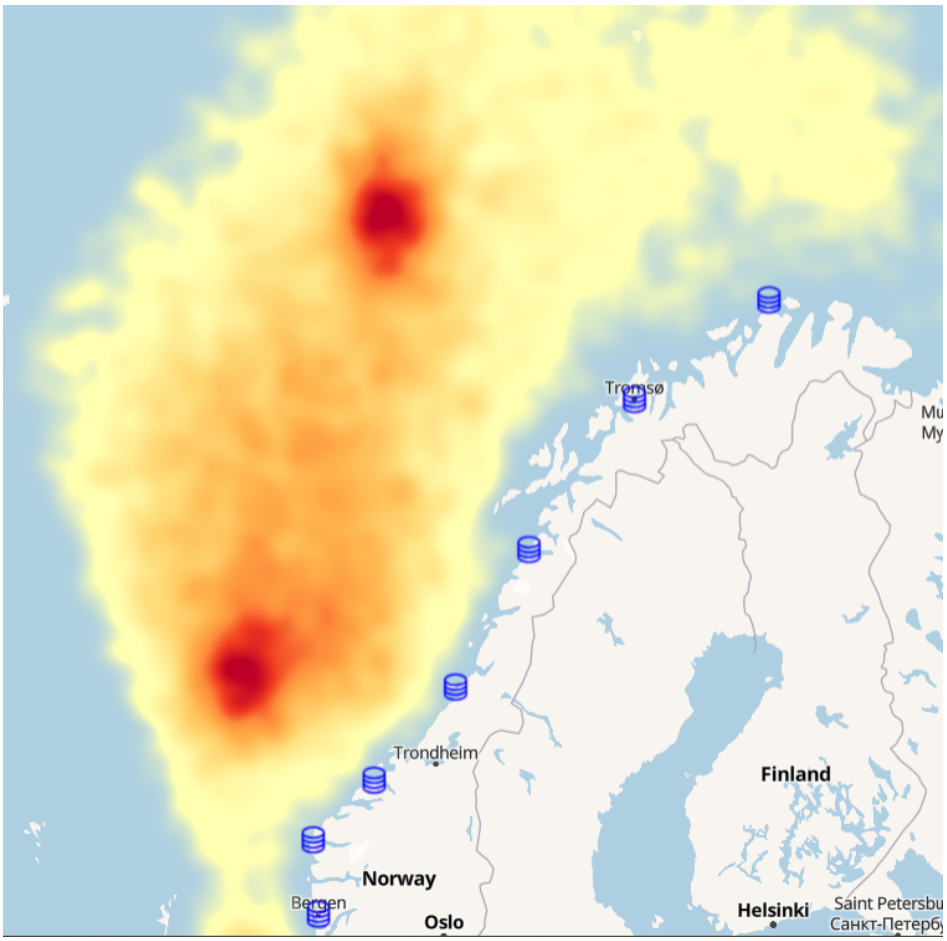


Figure 4.5.2: Heat map visualization showing anomaly density across the dataset, with warmer colors indicating higher concentrations

In Figure 4.5.2, the highest-density hot spots appear as dark-red areas over central Norway, north of Trondheim, and over southern Norway near Bergen, while the surrounding coast and inland areas show lower, more diffuse density in orange and yellow. The heat map view updates dynamically during zoom and drag operations, with zoomed-out levels producing higher aggregates per pixel radius and a denser overall representation.

4.5.3 Base Station Visualization

A base station layer is part of the application, implemented as mentioned in Section 3.5.3. The base station layer is visualized through blue icons in Figure 4.5.3. The base station layer itself is persistent, meaning that it is a visualization layer that will always be present in the app, even if other methods of visualizing the data points are changed.

Figure 4.5.3: Base station layer showing fixed AIS receiver locations as blue icons, providing persistent geographic reference points on the map

Base stations are linked to anomalies through backend data representing transmissions from individual anomalies. This is then used in other layers to connect base stations to anomalies through signal strength. The base station data itself will never change in position, as they are static points on the map, providing a persistent geographic reference between an anomaly and its transmitting base station.

4.5.4 Anomaly Group Visualization

The anomaly group visualization is shown in Figure 4.5.4.

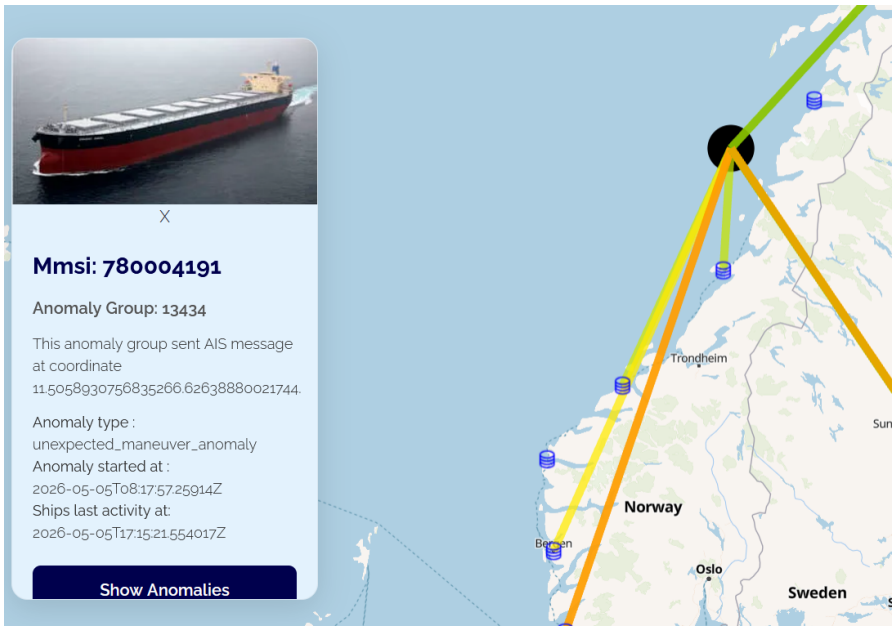


Figure 4.5.4: Anomaly group layer showing signal strength lines connecting individual anomalies to nearby base stations and satellites

The layer shows an info component for the current group, with coordinates, start date, and anomaly type. Signal strength is visualized through lines, connecting the individual anomalies inside the group to the base stations shown in the picture as blue symbols. The info component allows for users to show anomalies inside the group, through button press on the component, leading to the visualization being changed to the individual anomaly visualization.

4.5.5 Individual Anomaly Visualization

Individual anomalies are shown as a scatter plot (Section 2.7.1). The visualization shows the points colored based on signal strength, from red through yellow to green.

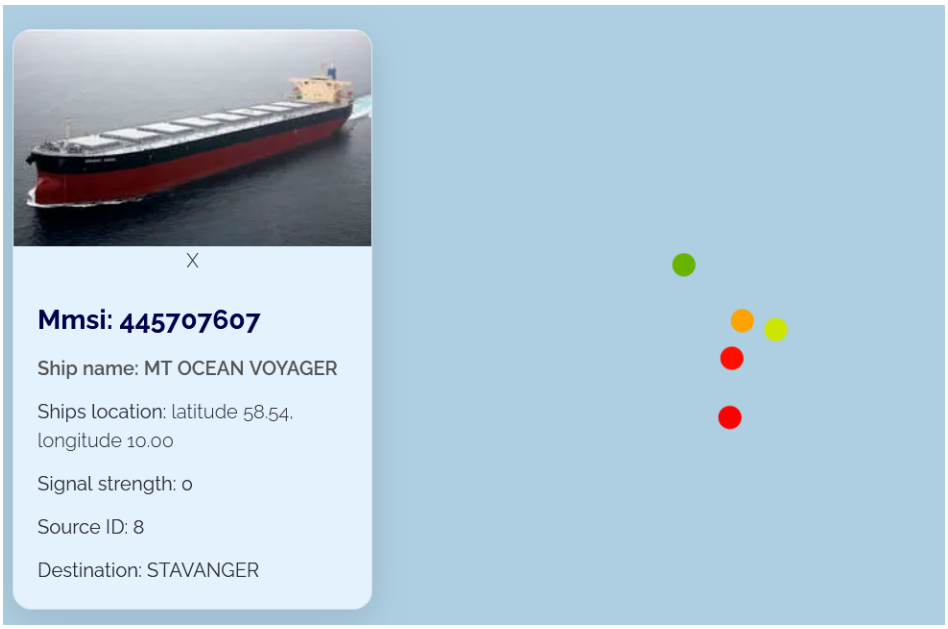


Figure 4.5.5: Individual anomaly layer displaying anomalies as a scatter plot, colored from red through yellow to green based on signal strength

Figure 4.5.5 shows the completed layer. Anomalies are correctly colored through the signal strength variable, with an info component being shown for an anomaly, through clicking on it. The info component illustrates features of the anomaly, like its direction, name, and MMSI. The point radius scales dynamically with the current zoom level so that anomaly points remain visually proportionate as the user zooms in and out.

4.6 Filter and Interaction

This section presents the delivered filtering and interaction features, implemented as described in Section 3.6.

4.6.1 Filter System

The filter system allows the rendered dataset to be narrowed by anomaly type, MMSI, and a start and end date, as shown in Figure 4.6.1.

Figure 4.6.1: Filter component showing the anomaly type selector, MMSI input, and date range fields used to narrow the rendered dataset

4.6.2 Temporal Filtering

A date-time range selector allows the visualization to be restricted to a chosen time period. The selector reuses an existing expandable React calendar component, which diverged from and simplified the date picker shown in the wireframe, as shown in Figure 4.6.2.

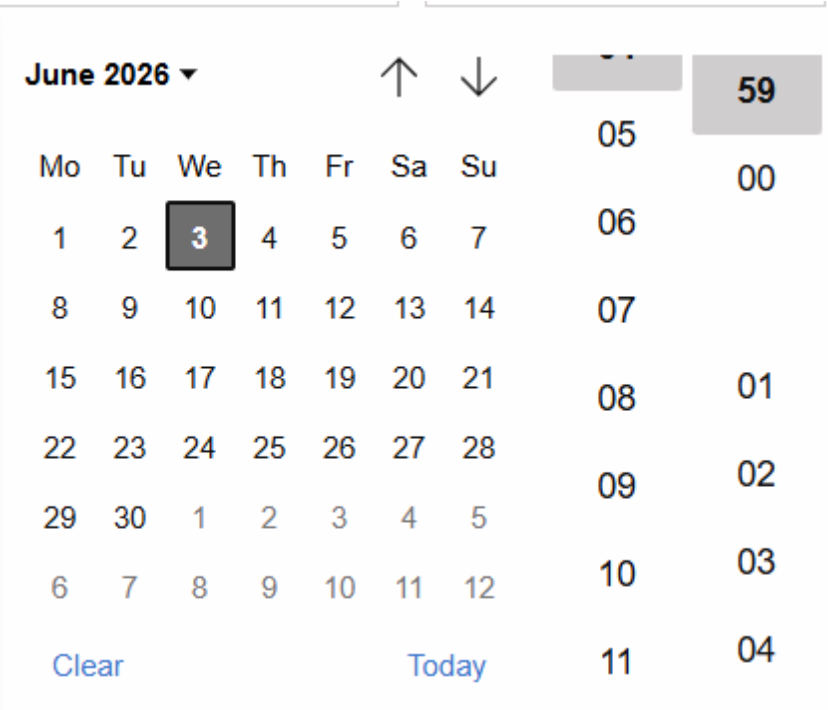


Figure 4.6.2: Date picker calendar component allowing operators to select a start and end date for temporal filtering

When a date range is applied, the map updates to display only the anomaly groups which were active within the selected period, while the sidebar reflects the same filtered dataset that is rendered. Both the start and end date can be set independently, which allows operators to examine anomaly patterns within any time window available in the dataset.

4.6.3 Satellite Visualization

Satellite connections are displayed as a layer that appears only when a data point is clicked, as shown in Figure 4.6.3. The satellite-orbit API is access-restricted and requires credentials, so relying on it would prevent others from testing this public prototype. Each satellite point is therefore approximated by finding the closest position in time within a 24-hour window of the anomaly. The 24-hour pass data is stored as a static JSON file bundled with the frontend and searched in memory at runtime, requiring no network call.

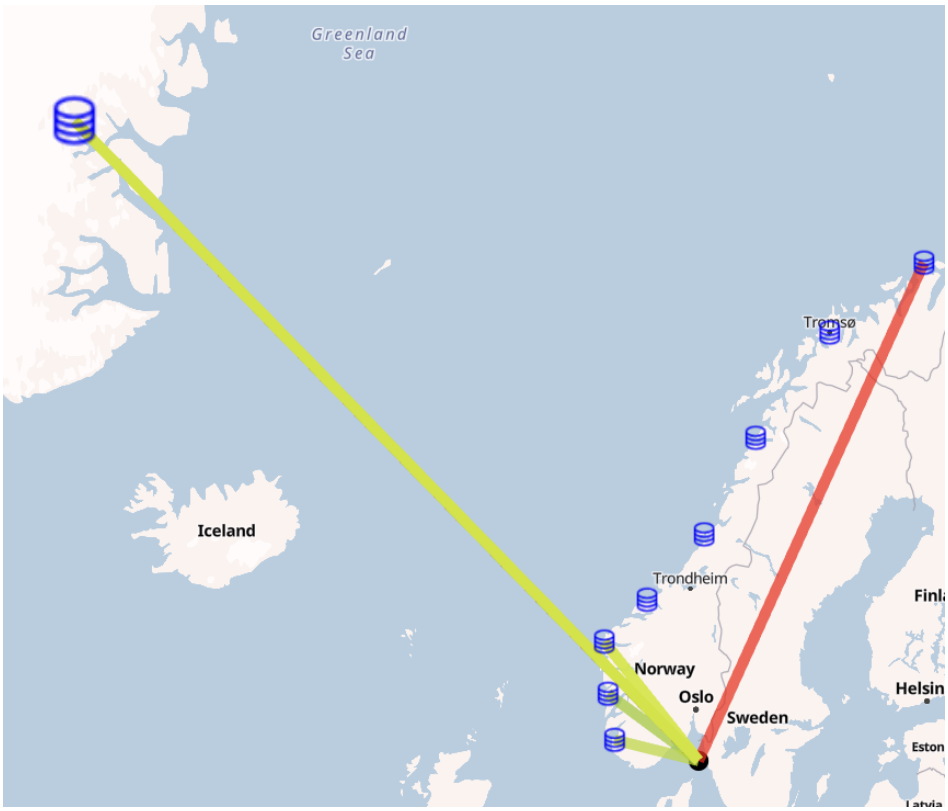


Figure 4.6.3: Satellite connections displayed when clicking a data point, showing each satellite approximated to its closest position within a 24-hour window of the anomaly

4.6.4 Interactive Components

The interface provides interactive controls for selecting filter values: a pop-up calendar component for the date range, a dropdown for selecting the anomaly type, and a numeric input for MMSI. In addition, a visualization toggle switches between the clustering and heat map layers, and a search bar above the sidebar filters the already-loaded dataset on the client side, without sending a new request to the backend.

4.7 Testing and Performance

Testing was done manually, as mentioned in Section 3.7. Console logging and observation based testing was used, to ensure all features work, with performance testing being done through use of the application.

Interaction features such as clicking individual data points were each verified on at least two separate points, to confirm they behaved consistently across the dataset. Backend endpoints were checked through the Swagger UI and by logging the returned data in the browser console, confirming that filtered requests returned the expected data. Testing also surfaced the need for the master query: filtering the full dataset on the frontend proved inefficient, which led to the decision to query the filtered data directly from the backend instead (Section 3.4).

Performance was observed during normal use of the application, primarily at the default zoom level with two to three page refreshes per observation, on the two laptops used by the group for ordinary schoolwork. Each visualization type was observed

at different zoom levels and across different areas of the map. The application was tested with a main dataset of 10,000 points, and additionally with larger datasets of 100,000 and one million anomalies generated through backend configuration files. Backend API response times were recorded from Docker logs and are summarized in Table 4.7.1. For the 100,000-anomaly dataset, unfiltered requests returned in 196–461 ms, with the higher end reflecting cold requests before database caching took effect. Applying a single anomaly type filter reduced response times to 55–62 ms. For the one million anomaly dataset, unfiltered requests took 2.0–3.1 s, while filtered requests returned in 592–920 ms.

Dataset size	Groups	Unfiltered response	Filtered response
100,000 anomalies	15,331	196–461 ms	55–62 ms
1,000,000 anomalies	153,910	2.0–3.1 s	592–920 ms

Table 4.7.1: Backend API response times for the anomaly-groups endpoint across two dataset sizes, measured from Docker logs. The unfiltered range reflects initial cold requests versus subsequent requests benefiting from database caching. Filtered results used a single anomaly type filter.

4.7.1 Rendering Performance

The rendering performance is responsive, but will vary based on graphics processing speed of differing computers (Section 2.10). Rendering happens first with the base map, then with data points, with the other user interface parts appearing with the map. Different visualization methods have different performance, with clustering being faster than heat maps in all cases.

4.7.2 Effect of Clustering on Performance

Clustering had a significant effect on the rendering performance of the application. Clustering renders fewer visual points on screen at any given zoom level, while the full dataset remains loaded. Due to data points being rendered only as they are in view, clustering leads to fewer points being shown while zoomed out, with zooming in progressively splitting clusters into smaller ones, while remaining less cluttered than a plain scatter plot would be.

4.7.3 System Responsiveness

Responsiveness of the system is observed to be at a satisfactory level, although some operations cost more than others, such as going from a clustered view to an anomaly group view, with signal strength lines being computed, causes some slowdown, but remains responsive after the layer is changed. Other parts of the system like filtering and changing of layers are responsive, and change as soon as input is submitted. The slowest layer is heat maps, as changing zoom or moving around the map, causes it to recalculate the weight of each area. The heat map itself is as responsive as Deck.gl implemented it, leading to no considerable performance gains to be doable, outside of trying to implement a clustered heat map in some way.

4.8 Customer Feedback

Feedback was collected from two sources through the process described in Section 3.8: the product owner at Kystverket, and an internal user who works with the AIS analysis system that is closely related to this project.

4.8.1 Product Owner Evaluation

The product owner evaluated the delivered features across four areas: clustering, signal strength visualization, heat map, and the solution as a whole.

4.8.1.1 Clustering

The clustering visualization was assessed positively because it helped provide a proper overview over a large amount of data without the visual clutter. The product owner also noted that clustering is already utilized by other large data systems at Kystverket, however it has not yet been applied to the AIS anomaly monitoring. A suggestion that was given for further development was to be able to display the types and counts of anomalies within each cluster. This would allow operators to assess if further exploring a cluster is worthwhile quickly.

4.8.1.2 Signal Strength

The signal strength visualization was described as a concept they were very interested in. The product owner noted that low signal strength between a vessel and a base station is an error in AIS messages that causes the creation of anomaly messages. Other systems they utilize for signal strength take longer to find the data and do not visualize it properly, increasing the time to analyze anomaly groups and the time complexity for the operator. The delivered solution was assessed as effective both at the macro level, where lines from anomaly groups to base stations are visible while zoomed out, and at the micro level, where individual signal strength values are visible when zoomed in. A suggestion was given to extend the visualization to include additional data like signal strength before and after an anomaly occurs, however this would require additional data from Kystverket.

4.8.1.3 Heat Map

The heat map was assessed to be a useful tool that complements clustering. This was because clustering only shows volume through numbers for large areas and requires zooming in to find areas with large amounts of points, while the heat map is valuable since it allows identification of all the hot spots on a large map at a zoomed-out level without requiring additional work from the user.

4.8.1.4 General Solution

The overall solution was evaluated positively for its features like toggling between visualization types, filtering by anomaly type, and the ability to drill down to the individual anomaly level. The product owner also noted that reimplementing anomaly group and base station views was necessary for the signal strength concept to provide value, even though those views already exist in Kystverket's current system.

4.8.2 Internal User Feedback

An internal user of Kystverket's existing AIS analysis system provided additional suggestions for further development.

- Normalizing the heat map against vessel traffic density in each area, so that the visualization reflects where anomalies are overrepresented rather than where traffic volume is highest.
- Extending signal strength visualization to cover all position reports from a vessel, not only those linked to anomalies, to provide a more complete picture of signal quality over time.

- Disabling clustering at low data densities where consolidation removes useful detail from the view.
- Adding aggregated per-vessel statistics to support long-term credibility assessment of individual ships.
- Displaying anomaly frequency graphs over time to reveal how the situation develops hourly or daily.

4.9 Project Outcome

The final project outcome was influenced both by the original project outline and by scope changes introduced throughout development. The implemented system follows the main goals of the original project outline, but has different features implemented as a result of the change of scope.

4.9.1 Scope Changes During Development

During development, and from the start of the project, the project differed in scope from the project outline in subtle ways. All visualization methods that the group worked on were in the project outline, but during the project itself the focus of the application became testing new concepts, such as signal strength visualization, through lines going from anomaly to base station/satellite came later during the project lifecycle. Features such as report generation were not mentioned later in meetings or as a feature request, rather other features were chosen for concept testing. Throughout the project, the customer gave requests on what would be good to implement at a given point in time, to test whether value could be gained from a full implementation in their own software.

4.9.2 Delivered vs Planned Features

Almost all planned features from the project outline and through meetings were implemented, some being seen as extra features later in development by the customer. A report generating system and satellite orbit visualization feature were not delivered. However satellites themselves were implemented and shown as connected to anomalies through a signal strength line in the anomaly group layer, showing its position at the time of data transmission. The overall delivered features implement the majority of the project outline's features, while also implementing the requested features from the customer.

Feature / Objective	Status	Notes
Clustering visualization	Implemented	
Heat map visualization	Implemented	
Base station layer	Implemented	
Anomaly group layer	Implemented	Reimplemented to support signal strength
Individual anomaly layer	Implemented	Added late at customer request
Filtering (type, MMSI, dates)	Implemented	
Signal strength visualization	Implemented	Added during the project; not in the original outline
Temporal analysis (forward/backward in time)	Partial	Delivered as a date-range filter only
Hexagonal binning	Not implemented	Cut due to time constraints
Report generation	Not implemented	Not requested again after early meetings
Satellite orbit plotting	Not implemented	Satellites shown only at transmission time

Table 4.9.1: Delivered versus planned features and objectives, including features added during development and shortfalls against the original Introduction objectives.

CHAPTER

FIVE

DISCUSSION

This chapter evaluates the finished project and reflects on the technical, methodological, and contextual factors that shaped the outcome.

5.1 Overview of End Product

The finished product meets the main requirement the customer requested, with few features from the original project description being omitted. The quality of the finished project as a concept test for new visualization methods was noted in the customer feedback presented in Section 4.8.

The overall solution was assessed as satisfactory by the customer, as it allowed them to test concepts of new visualization modes like signal strength, clustering, and heat map layers. Collaboration with the customer was effective throughout the project, with frequent communication supporting the addition of new features and the resolution of technical questions as they arose.

The project demonstrates that visualization of large datasets through methods not currently in use by the customer can provide concrete value. This is reflected in the customer's stated intention to incorporate several of these features into their existing software.

5.2 Comparison to the Customer's Existing Tools

As the resulting application tests concepts for the customer, a direct comparison with their existing tools yields little value, as the customer themselves noted that the tools currently in use are superior in their core functionality. The value of the delivered prototype instead lies in testing new visualization methods and features, allowing the customer to evaluate whether their existing software, or software they develop themselves, should incorporate these features. In the specific role of testing new concepts, the prototype therefore provides value that the customer's existing production tools do not.

5.3 Discussion of Technical Solution

The technical solution, described in Section 3.2, is documented and functions as intended, with emphasis on simplification through the use of helper classes and repeated refactoring throughout the project.

5.3.1 System Architecture

The system architecture, with Deck.gl overlaid on MapLibre as the base map and a Docker-based backend providing data, worked well throughout development. The modular approach led to MapLibre being separated from the Deck.gl implementation (Section 2.15.2), with the map itself responsible only for map-related functionality such as zooming and movement. This separation simplified the overall complexity and produced cleaner code with a clear separation of concerns. Achieving this required a period of refactoring tightly coupled code, particularly where the map itself contained functionality unrelated to its task.

The architecture choice was partially guided by the customer, whose request to use React and other technologies already in use within their infrastructure shaped the group's decisions, alongside with the customer providing the backend. Maplibre was suggested in the project document and was ultimately selected as the map provider, supported by its open-source nature and use of vector tiles (Section 2.8.1).

5.3.2 Frontend

The frontend implementation successfully followed the wireframes established at the beginning of the project (Section 3.3.3). The design is minimalist by comparison to other applications, prioritizing the map and visualization components over additional interface elements.

The choice of map provider had both advantages and disadvantages. MapLibre is a fork of the now closed-source Mapbox, which meant that documentation was less comprehensive than required, and that more detailed information for equivalent code often had to be retrieved from the Mapbox website. As a result working with MapLibre required more time spent understanding the library than was originally anticipated, delaying implementation beyond what had been planned.

Deck.gl (Section 2.15.2) proved to be an effective choice for visualization, providing all functionality needed to implement the customer's requested features. Layering through Deck.gl allowed straightforward integration with MapLibre by utilizing a MapBox-style overlay. Since MapLibre is a fork of MapBox, no additional integration code was required beyond what would have been needed for MapBox itself.

The use of React with TypeScript enforced stronger type safety than plain React, but introduced an initial learning curve. The group's lack of experience with the coding standards needed for react and typescript, led to development time being spent on learning the basics as well as more advanced tools such as Context, which was ultimately used to simplify the code.

The single-page design, which covered all required functionality through interactive components, suited the project well. One advantage over a multi-page architecture, was that the map and visualization remained visible at all times, including during manual testing of other components. The overall simplicity of the interface also aligned with the project's focus on concept testing, particularly as the customer themselves had requested that less time would be spent on design and more on

functionality.

Overall, the frontend implementation fulfilled the project's main goals while highlighting how the use of third party tools can present challenges, and how tool selection is an important step in any software development project.

5.3.3 Code Quality and Maintainability

The code quality was designed to follow software principles, but due to lack of experience with React itself, some uncertainties about use of specific methods arose. The use of prior programming experience to create files that handle specific tasks, are however used to simplify the program, with refactoring being done multiple times to follow these guidelines

5.3.4 Visualization Methods and User Interaction

The chosen visualization methods were effective for different aspects of anomaly visualization, each with their own strengths and weaknesses related to readability, performance and interaction.

Clustering, implemented as described in Section 3.5.1 and shown in Section 4.7.2, produced a measurable performance improvement compared to a normal scatter plot of all points in the dataset, while also improving readability.

The zoom function with layer recalculation was observed to remain responsive during manual testing, although responsiveness remained dependent on the hardware running the application. The layer was implemented with a focus on simple visualization, including a numeric label indicating count of anomaly groups within each cluster, presented in a clear and readable manner. The layer is also interactive, with pressing a single individual anomaly group cluster leading to the anomaly group layer, this is performant through API call to the backend, instead of acquiring the information through frontend filtering. The main drawback with clustering is the lack of specific details of each anomaly group, requiring multiple layer changes to go into an anomaly group and then into the individual anomalies, it gives a generally simpler overview of where anomalies are grouped, and how numerous they are, but no specific details on the anomalies inside. Although this is worth the tradeoff versus having a normal scatterplot with a chaotic amount of points shown at once.

The Heat map visualization, implemented as described in Section 3.5.2, provides value by representing the entire dataset, rather than the aggregated form used by clustering. This produces a overview of all data points based on proximity, in contrast to clustering visualization, it utilizes color intensity to represent hot spots more precisely. Color based encoding is more visually intuitive for identifying hot spots compared to visually identifying density based on size and numeric labels. This benefit also justifies higher rendering costs utilized by heat map's. This means the visualization method is slower than other methods while it offers a more comprehensive overview of the dataset.

The anomaly group layer, described in Section 2.3, provides detailed information about each individual group. The signal strength visualization is central to this layer, and the implementation received positive feedback from the customer (Section 4.8). The combination of location, anomaly type, and connections to base stations and satellites in the information component,provides a clear overview of how an anomaly group relates to the receivers of AIS messages.

The individual anomaly layer, implemented as a scatter plot (Section 2.7.1), was

added at the customer's request, with an info component being used to display the data properties of each anomaly. The layer provides further insight into the individual anomalies within a group, including their heading and signal strength. The layer adds clear value to the project and presents no significant drawbacks.

The remaining visualizations, including the base station layer (Section 3.5.3) and the satellite layer, are required for layers such as the anomaly group with signal strength to provide any useful information. As static reference points, the base stations also give operators visual feedback on the spatial relationship and distance between anomaly clusters and the base stations that received their transmissions. They are therefore crucial to the project and present no notable drawbacks in their implementation.

User interaction is a core element of the application, and users should be able to navigate and retrieve information without noticeable slowdown or errors. Based on manual testing during development, user interaction was observed to remain responsive, even though performance remains heavily dependent on hardware used, with the available GPU acting as the main limiting factor for rendering performance (Section 2.10).Most user interaction centers on zoom level and camera position within the map. Although this map interaction has performance costs, it enables access to anomaly information, filtering and searching. As interactivity is a core feature of the application, this trade off is acceptable, since dynamic datasets are far easier to analyze interactively than statically.

5.3.5 Backend and Data Handling

The backend was provided by the customer with most of the endpoints required by the project already implemented, supported by comprehensive Swagger documentation. Docker simplified the process of running the backend, requiring only a few commands to start and fill the database with test data.

The group's main contribution to the backend was a master query endpoint (Section 3.4), which combines all filter parameters into a single backend query and returns only the matching dataset. Client-side filtering on top of an already-filtered dataset would have been possible, however that solution would have required keeping the original dataset in memory and rerunning the filter logic on the frontend to further filter the data the group already has. However that would be inefficient at the dataset sizes used in this project, because the containerized backend responds in the millisecond range and routing filter operations to the backend to then be handled by the query proved faster than client-side re-filtering. Because of this the rendered dataset always reflects the currently active filters directly, without the need to keep a separate ”full dataset” on the frontend. When users need to drill down further within the active dataset, the sidebar search component provides this functionality locally without requiring an additional backend request.

Observation of the backend showed no critical performance or data errors. However, working with the delivered backend introduced limitations, on being constrained to the predefined API structure, with extensions being more complicated for the group to implement themselves, due to lack of experience with the programming language GO. Although the observed performance was satisfactory, testing of the performance was not controlled by the group itself, with core negatives of the backend implementation being wait time for customer to implement required features, a less often occurrence. As a result the backend was well implemented, but not controlled solely by the group, this being a tradeoff.

Data handling through GeoJSON, proved a good choice for both the backend and frontend, with the libraries used already supporting the data. The data itself being parsed through a deck.gl layer. The layer being responsible for parsing data from each GeoJSON feature. Proved to be an effective design, that allows for simple access to data, through accessing properties and geometry of said features. A trade-off of using GeoJSON is that the frontend must have prior knowledge of the data structure in order to correctly access said properties and geometry. This introduces dependency on a clear schema. Another issue with data, has been asynchronous errors, with wrong data being delivered, or data being accessed before delivery.

5.4 Performance and Scalability Discussion

The performance of the system is highly dependent on hardware specifications, with the GPU having a particularly strong influence. WebGL benefits significantly from more capable GPUs (Section 2.10), meaning that performance is bottlenecked on weaker systems.

5.4.1 Rendering Performance

The base rendering performance of MapLibre was observed to be unproblematic, with performance being taxed on the visualization methods and dataset size, rather than the map. Datasets ranging from a ten thousand to over 1 million points were tested. The architectural design of the project is not bottlenecked based on the backend, but with frontend using supercluster (Section 2.15.5) to recalculate clusters based on zoom, and similar calculations being done on heat map, leads to zoom and move operations being most taxing on the system. This can be tweaked to recalculate fewer times on zoom level, but will lead to a less responsive system. The core bottleneck of any performance is then the frontend itself, as the backend response time scales with dataset size from under 500 ms at 100,000 anomalies to 2–3 s at one million anomalies for unfiltered requests (Table 4.7.1).

5.4.2 System Responsiveness

The backend showed acceptable response times using the primary dataset size of 100,000 anomalies, with unfiltered requests returning in 196–461 ms and filtered requests in 55–62 ms (Table 4.7.1). The higher end of the unfiltered range was still interactive in practice, because the system does not render all data points at once. Since points are only rendered as the user zooms past the clustering threshold. The robust backend also helps with subsequent queries because of database caching to make it seem more interactive.

For the one million anomaly dataset, unfiltered requests took 2.0–3.1 s and filtered requests returned in 592–920 ms. For intended use cases such as a concept prototype evaluated at the customer’s data volumes, the 100,000-anomaly figures are more representative of expected operational conditions.

On the frontend, the most costly interaction observed was switching from the clustered view to the anomaly group layer, where signal-strength lines are computed for each anomaly in the group. Other interactions, including applying filters, switching between clustering and heat map layers, and searching in the sidebar, responded immediately on both development laptops used during testing. The heat map was the slowest visualization layer overall, since it recalculates area weights on every zoom and drag event. This is bounded by Deck.gl’s implementation and cannot be

improved further without adopting a fundamentally different approach such as a pre-aggregated heat map.

5.5 Project Execution

This section evaluates the project management process described in Section 3.1.1, focusing on how the chosen methodology, team structure, and communication practices influenced the development outcome.

Although the adopted methods ensured a structured development process, several of them introduced weekly overhead that proved disproportionate for a two-person team operating under a fixed deadline. The following subsections evaluate each aspect individually.

In retrospect, the team should have engaged more carefully with the project requirements and assessment criteria at an earlier stage, because it would have improved prioritization throughout the project. Clearer focus on which deliverables were most valued would have allowed the group to direct development time more effectively, which should have been a main focus given the reduced team capacity. This is a lesson that would be applied from the start in future projects of similar scope.

5.5.1 Scrum Framework and Sprint Structure

The Scrum framework provided a consistent structure for planning and delivery. The system was designed for large groups, so it had issues in relation to workload and redundancy for groups of two. Therefore, the daily stand-up system was altered and implemented to occur multiple times a day instead to make sure the group had proper accountability and focus in relation to scope. It also helped keep motivation high and allowed the group to quickly make changes to tasks depending on problems found with them. Instead of the standard two-week sprint, the group used weekly sprints for the majority of the project, and only utilizing two-week sprints for the final two cycles (weeks 15–16 and 17–18), because the focus shifted to report writing. The first two sprints were documented informally because the official sprint templates were not yet adopted, however from sprint three onward, planning, review, and retrospective documents were consistently maintained for all 13 sprints.

However, several Scrum artifacts introduced documentation overhead that provided limited value at this team size. This became a bigger issue because of the decision to have weekly sprints, which increased the overhead compared to the standard two-week cadence. The problem was amplified because sprints documented and reviewed the backlog every week, even though group members had continuous visibility into task status through GitHub Projects.

In relation to sprint documentation, the sprint artefacts provided traceable project documentation, which served as a reference when writing the report while ensuring important information was not lost in miscommunication.

5.5.2 Team Composition and Workload Distribution

The project began with three members but continued with two after one member left early in the development period. This reduced team capacity raised the per-member workload and directly altered the scope decisions made with the customer. Because of that the visualization features offering the most prototype value were prioritized,

while report generation and hexagonal binning were deprioritized. The daily stand-ups became more important in this context, because they ensured both members remained focused and that problems blocking progress were resolved quickly rather than accumulating into a pile of more issues.

In a group of two where both members had mostly the same knowledge and expertise, workload was distributed based on member preference and the nature of each task, which made documentation responsibilities formally split where one member handled the sprint documentation while the other handled meeting documentation while development tasks were shared. In practice however, more demanding tasks were rotated between members to maintain motivation. This type of informal balancing worked well for a group of 2 but would not scale to a larger team, where responsibilities would be split more evenly with explicit role definitions.

5.5.3 Communication and Meeting Structure

Slack provided an efficient channel for direct communication with the customer, enabling rapid resolution of short technical questions. The product owner's written evaluation noted over 250 messages exchanged through Slack by the 11th of May, and acknowledged that the group took initiative in arranging progress meetings to present ongoing development. The customer's domain expertise allowed complex questions to be resolved within short discussions rather than through a long discussion. While communication with the supervisor was handled primarily through email, with in person consultations used for urgent issues, because most work was conducted on campus.

For meetings, biweekly meetings were held with both the supervisor and the customer at Kystverket. Before these meetings, discussions were held to discuss which questions should be asked. However, most of these discussions happened through a small meeting before the actual meeting or through daily stand-ups within the group.

Most meetings with the customer were focused on whether the customer liked what had been developed, followed by technical questions to figure out the difficulty of harder problems and what they wanted the group to focus on. With the supervisor, however, problems relating to documentation and any issues with what had been done were mostly discussed. There were sometimes occasions where meeting times were delayed because the customer was busy or had taken a break from work.

5.5.4 Task Management and Version Control

The task management infrastructure described in Section 3.1.6 worked effectively for the project, although the multi-repository setup introduced some context-switching overhead. The four workflow stages would have been more useful in a larger group, however since work was handled in person they served mainly to track which tasks had not been done or which member was currently working on each task. The three-repository structure occasionally caused issues to be filed in the wrong board, where they were overlooked until rediscovered and relocated. The burndown chart was generated automatically and used to track project progress and identify whether the project was accumulating unaddressed issues. The three-board structure worked better than a single combined board, because GitHub Projects allowed all three to be viewed together at the overview level while keeping each repository's issues separated at the detail level.

For version control, Git was utilized because it allows code to be reverted easily if

new code breaks the system or creates unexpected issues. Branching was useful overall, even though it occasionally caused merge conflicts and required refactoring after code was combined. This was still preferable to working in a single shared branch, where both members would have caused conflicts continuously during development.

5.5.5 Scope Management

The reduction from three to two team members early in the project heavily influenced scope decisions. Because of the large change in the amount of workload the group could handle the customer agreed that the team would prioritize the visualization features that offered the most prototype value. The features that offered the most value were: clustering, heat maps, signal strength visualization, and filtering, however the customer also wanted the group to deprioritize report generation, hexagonal binning, and satellite orbit plotting. This was a deliberate decision based on the groups capacity rather than an unplanned shortfall, while keeping the delivered system focused on what the customer most needed to evaluate as a concept prototype.

Scope management was handled reactively through customer meetings rather than through a formal change process, which had both benefits and drawbacks. The weekly sprint cycle allowed scope changes to be incorporated quickly, which was valuable given the short development period and limited team capacity. This approach kept development aligned with the customer's interests and prevented time from being spent on challenging features that were not needed. However, the absence of a formal scope freeze meant that new requests led to planned features being cut to accommodate them. The reactive approach worked well within the constrained timeline, although a longer project would have permitted more features to be properly developed and polished. For a project of similar length, a formal scope freeze date would improve predictability. Weighted prioritization could also help larger teams manage competing requests, but the small team size and direct communication with the customer made this less necessary in this project.

5.6 Limitations

Although the project fulfilled the main requirements as adjusted through customer-requested scope changes, several limitations remain which reflect its role as concept-testing software.

5.6.1 Technical Limitations

One technical limitation is that real AIS data from the customer was not integrated into the project. This integration would not have been feasible within the scope of the project, as the data is classified which makes it a task that would need to be handled internally. Another limitation is the absence of satellite orbit plotting, which was requested but could not be implemented within the available time. This feature would have allowed users to view a satellite's orbit from its current position to its future projected positions. Although satellite orbits were not implemented, satellites themselves are present in the system and appear in the signal strength visualization. A further feature from the original project outline that was not implemented is automatic report generation. This was removed from the scope during the project as focus shifted toward concept testing.

CHAPTER

SIX

CONCLUSIONS

This chapter summarizes the project, revisits the objectives, and reflects on the overall outcome of the developed system

The goal of the project was to evaluate whether alternative visualization methods and features could provide new value to the Norwegian Coastal Administration, in analyzing AIS anomaly data. The focus was on developing a prototype application that enables analysis of large-scale anomaly data sets through interactive filtering and multiple new visualization approaches.

The final system demonstrates a locally hosted prototype allowing for analysis of AIS anomaly data. It allows users to view anomaly patterns through clustering, heatmap visualizations and detailed inspection of anomaly groups and individual anomalies. In addition, filtering based on MMSI, anomaly type and date range were added to enable a targeted analysis of anomalies and their connection to base station and satellites. Together, these features support a high level view of where anomalies aggregate, and their signal connection to base stations on the coast alongside satellite representation.

The project demonstrates that it is possible for third party tools and modern web technologies to prototype new features and methods in data analysis. This approach enables early evaluation of visualization concepts without requiring full scale development on these systems, allowing verification of the concepts value and reducing development risk, and unnecessary time usage.

The project objectives were in large part achieved, fully implementing the core visualization and interaction features requested, including clustering, heatmaps, filtering and information components, alongside signal strength and individual anomaly layers. However, some planned features, such as hexagonal binning, more complex visualization of satellite orbits, and report generation, were not implemented due to time constraints and scope cuts that kept focus on the delivered system.

Further improvement could be implemented, such as integration with the customers real-time AIS data stream, to enable monitoring of live data. More advanced temporal analysis or implementation of cut features such as hexagonal binning, can add new value to the system. The system can also be further worked on to visualize

more than AIS data, and can be used to test concepts related to other fields and visualization methods.

Overall, the implemented system meets the main objective of creating a simple, readable visualization program for AIS anomaly data, while remaining open for further work.

CHAPTER

SEVEN

SOCIETAL IMPACT

This chapter discusses the societal and economic consequences, or lack thereof, of the project, while discussing sustainability and environmental aspects.

7.1 Societal and Ethical Impact

During discussion with the customer regarding societal and ethical impact, several key insights were obtained. Better visualization of AIS anomalies is a tool that would transform current workflows rather than eliminating employment opportunities, creating a system where the use of more effective tools enables more specialized roles. As such, the workplace impact of a fully developed software solution would not be negative, and the use of better tools would not lead to loss of employment.

Because the system primarily analyzes data from large vessels, smaller private vessels would not be affected. As a result, no surveillance of individual persons would occur if the tool were connected to real data. Furthermore, commercial vessels are required by law to use AIS transponders [2]. The system therefore does not introduce a new form of surveillance; instead, it acts as an analysis layer over existing data.

Another point raised in communication with the customer is around the scenario of whether this could be used to stop illegal smuggling or immigration. However, the customer considered this unlikely, as the system targets large vessels, which are less likely to be associated with or suited for illegal activities such as smuggling.

A notable ethical consideration is that anomaly detection can contain errors or false positives, which could lead to unnecessary investigation of non-anomalous vessels [54]. This risk is particularly high if the tool is used in decision-making rather than as an aid for human operators identifying anomalies.

7.2 Economic

Software prototyping is widely recognized as a development approach that helps reduce cost and risk by gaining early information about issues as well as user feedback before committing to a final implementation [55]. The prototype created by the group demonstrates how feature testing can be used to evaluate the system's

actual value. This reduces financial risk by avoiding implementations of features that do not provide sufficient value.

The prototype the group created relies entirely on open-source tools (such as React, MapLibre, Deck.gl, Supercluster, and Docker), meaning that it has zero software licensing costs, in contrast to commercial visualization platforms. Instead of investing large costs into a project, the investment is in development time to create prototypes that allow the customer to evaluate a feature's value before committing to an actual full production build in their internal systems. The cost of a short prototyping phase is minimal compared to the extensive resources required to create and maintain a full-scale production feature that might prove unhelpful in the future. While prototyping can increase development time and effort, this is a mitigated downside compared to the value the prototype provides.

7.3 Sustainability

The direct sustainability impact of a software development project is generally lower than in physical engineering fields, because software does not require the same volume of raw materials and manufacturing. However, core sustainability factors such as energy-efficient computing, maintainable codebases, and technology choices remain highly relevant for reducing the carbon footprint of IT systems [56].

The group's project is a locally hosted prototype, resulting in a minimal environmental impact, as there is no material cost outside of personal computers. Because the prototype is used to evaluate the viability of concepts, it helps to make an informed decision whether to implement said concepts in the customer's current solution. This may contribute to a positive sustainability impact due to mitigating resource use on wasted features. The system itself is maintainable, while being able to be extended in functionality.

The project aligns itself with the United Nations Sustainable Development Goals [57], particularly Goal 9 (Industry, Innovation and Infrastructure) and Goal 14 (Life Below Water). Because the system utilizes a standard web stack and open-source tools to prototype new anomaly analysis methods, the project supports innovation in maritime monitoring infrastructure without the need for specialized hardware, which helps fulfill the objectives of Goal 9. Additionally, by improving how AIS anomalies are analyzed, it creates a system that supports more reliable detection of faulty or suspicious vessel behavior. This contributes directly to maritime safety and protection of the marine environment, which aligns directly with Goal 14.

7.4 Environment

Because the software does not require physical components aside from standard computers, its direct environmental footprint is relatively low. Electricity consumption during use of the software is the primary environmental factor. However, improved AIS anomaly detection could enable authorities to take faster action regarding faulty, hazardous, or drifting vessels, potentially leading to reduced severe environmental impacts such as oil spills or marine pollution. Therefore, while the software itself has minimal negative environmental impact, its application could yield substantial positive environmental impact.

REFERENCES

[1] Norwegian Coastal Administration. *What is the role of the NCA?* URL: https://www.kystverket.no/en/about-us/what-is-the-role-of-the-nca/ (visited on 05/28/2026).

[2] Norwegian Coastal Administration. *AIS Norway*. URL: https://www.kystverket.no/en/sea-transport-and-ports/ais/ais-norge/ (visited on 05/28/2026).

[3] BarentsWatch. *AIS Displays Status Report at Sea*. URL: https://www.barentswatch.no/en/articles/ais-displays-status-report-at-sea/ (visited on 05/28/2026).

[4] *AIS (Automatic Identification System) Overview*. NATO. 2021. URL: https://mc.nato.int/nsc/media-centre/news-archive/2021/ais-automatic-identification-system-overview (visited on 05/23/2026).

[5] IBM. *What is a digital twin?* URL: https://www.ibm.com/think/topics/digital-twin (visited on 05/22/2026).

[6] TarmacView. *Signal Strength*. URL: https://www.tarmacview.com/glossary/signal-strength/ (visited on 06/01/2026).

[7] IBM. *What is data visualization?* URL: https://www.ibm.com/think/topics/data-visualization (visited on 05/23/2026).

[8] Bo Zhang. *Computer vision vs. human vision*. 2010. URL: https://ieeexplore.ieee.org/document/5599750 (visited on 05/23/2026).

[9] Atlassian. *A complete guide to heatmaps*. URL: https://www.atlassian.com/data/charts/heatmap-complete-guide (visited on 05/23/2026).

[10] *London Heat Map: Informing a City's Decarbonisation Plan*. Centre for Sustainable Energy. URL: https://www.cse.org.uk/research-consultancy/consultancy-projects/london-heat-map-informing-a-citys-decarbonisation-plan/ (visited on 05/23/2026).

[11] IBM. *What is clustering?* URL: https://www.ibm.com/think/topics/clustering (visited on 05/23/2026).

[12] *K-means Clustering Visualization in R: Step-by-Step Guide*. Datanovia. URL: https://www.datanovia.com/en/blog/k-means-clustering-visualization-in-r-step-by-step-guide/ (visited on 05/23/2026).

[13] Atlassian. *A complete guide to scatter plots*. URL: https://www.atlassian.com/data/charts/what-is-a-scatter-plot (visited on 05/23/2026).

[14] MapBox. *Web Maps*. URL: https://www.mapbox.com/insights/web-maps (visited on 05/23/2026).

[15] MapBox. *Vector tiles introduction*. URL: https://docs.mapbox.com/data/tilesets/guides/vector-tiles-introduction/ (visited on 05/23/2026).

[16] *Working with Vector Tiles*. QGIS Documentation. URL: https://docs.qgis.org/3.44/en/docs/user_manual/working_with_vector_tiles/vector_tiles.html (visited on 05/23/2026).

[17] MapLibre. *About MapLibre*. URL: https://maplibre.org/about/ (visited on 05/23/2026).
[18] MapLibre. *MapLibre GL JS GitHub Repository*. URL: https://github.com/maplibre/maplibre-gl-js (visited on 05/23/2026).
[19] Amazon. *What is an API (Application Programming Interface)?* URL: https://aws.amazon.com/what-is/api/ (visited on 05/23/2026).
[20] IBM. *What is an API (application programming interface)?* URL: https://www.ibm.com/think/topics/api (visited on 05/23/2026).
[21] Professor Samuel Mbugua Geofrey Mwamba Nyabuto Mr. Victor Mony. "Architectural Review of Client-Server Models". In: *International Journal of Scientific Research and Engineering Trends* (2024). URL: https://www.researchgate.net/publication/376512127_Architectural_Review_of_Client-Server_Models (visited on 05/23/2026).
[22] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000. URL: https://roy.gbiv.com/pubs/dissertation/rest_arch_style.htm (visited on 05/23/2026).
[23] IBM. *What is REST API?* 2025. URL: https://www.ibm.com/think/topics/rest-apis (visited on 05/23/2026).
[24] ScienceDirect Topics. *Hardware Acceleration*. URL: https://www.sciencedirect.com/topics/computer-science/hardware-acceleration (visited on 05/25/2026).
[25] Mozilla. *WebGL: 2D and 3D graphics for the web*. 2025. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API (visited on 05/23/2026).
[26] David Voyles. *A Beginner's Guide to WebGL*. 2015. URL: https://www.sitepoint.com/beginners-guide-webgl/ (visited on 05/23/2026).
[27] The Open University. *Approaches to Software Development: Coupling and Cohesion*. URL: https://www.open.edu/openlearn/science-maths-technology/approaches-software-development/content-section-1.5.4 (visited on 06/02/2026).
[28] Edsger W. Dijkstra. "On the Role of Scientific Thought". In: *Selected Writings on Computing: A Personal Perspective*. Springer-Verlag, 1982, pp. 60–66. URL: https://www.cs.utexas.edu/~EWD/transcriptions/EWD04xx/EWD447.html (visited on 06/02/2026).
[29] Robert C. Martin. *The Single Responsibility Principle*. 2014. URL: https://blog.cleancoder.com/uncle-bob/2014/05/08/SingleReponsibilityPrinciple.html (visited on 06/02/2026).
[30] Cole Stryker Rina Diane Caballar. *What is code documentation?* 2025. URL: https://www.ibm.com/think/topics/code-documentation (visited on 05/23/2026).
[31] Martin Fowler. *Refactoring*. URL: https://refactoring.com/ (visited on 05/23/2026).
[32] Nielsen Norman Group. *10 Usability Heuristics for User Interface Design*. 1994. URL: https://www.nngroup.com/articles/ten-usability-heuristics/ (visited on 05/23/2026).
[33] Appnovation. *12 Principles of Data Visualization*. 2023. URL: https://www.appnovation.com/blog/12-principles-data-visualization (visited on 05/23/2026).
[34] Interaction Design Foundation. *What are Wireframes?* 2016. URL: https://ixdf.org/literature/topics/wireframe (visited on 05/23/2026).
[35] Figma. *What is wireframing?* URL: https://www.figma.com/resource-library/what-is-wireframing/ (visited on 05/23/2026).
[36] IBM. *What is Software Testing?* URL: https://www.ibm.com/think/topics/software-testing (visited on 06/02/2026).
[37] Microsoft. *What is Agile?* URL: https://learn.microsoft.com/en-us/devops/plan/what-is-agile (visited on 05/23/2026).

[38] Ken Schwaber and Jeff Sutherland. *The 2020 Scrum Guide*. 2020. URL: https://scrumguides.org/scrum-guide.html (visited on 05/23/2026).
[39] Sohrab Salimi. *The Daily Standup*. URL: https://www.agile-academy.com/en/scrum-master/daily-standup/ (visited on 05/23/2026).
[40] Scrum.org. *What is Scrum?* URL: https://www.scrum.org/resources/what-scrum-module (visited on 05/25/2026).
[41] Meta Open Source. *React Documentation*. 2025. URL: https://react.dev/learn (visited on 05/23/2026).
[42] Microsoft. *The TypeScript Handbook*. URL: https://www.typescriptlang.org/docs/handbook/intro.html (visited on 05/23/2026).
[43] CARTO. *Deck.gl*. URL: https://carto.com/glossary/deckgl/ (visited on 05/23/2026).
[44] deck.gl. *Learning Resources*. URL: https://deck.gl/docs/get-started/learning-resources (visited on 05/23/2026).
[45] deck.gl. *deck.gl Official Website*. URL: https://deck.gl/ (visited on 05/23/2026).
[46] Google Developers. *deck.gl Overlay View for Google Maps JavaScript API*. URL: https://developers.google.com/maps/documentation/javascript/deckgl-overlay-view (visited on 05/23/2026).
[47] json.org. *Introducing JSON*. URL: https://www.json.org/json-en.html (visited on 05/23/2026).
[48] Butler, Howard and Daly, Martin and Doyle, Allan and Gillies, Sean and Hagen, Stefan and Schaub, Tim. *The GeoJSON Format*. 2016. URL: https://datatracker.ietf.org/doc/html/rfc7946 (visited on 05/23/2026).
[49] Mapbox. *supercluster*. 2024. URL: https://github.com/mapbox/supercluster (visited on 05/23/2026).
[50] Mapbox. *Clustering millions of points on a map with Supercluster*. 2016. URL: https://blog.mapbox.com/clustering-millions-of-points-on-a-map-with-supercluster-272046ec5c97 (visited on 05/23/2026).
[51] docker.com. *Docker Overview*. URL: https://docs.docker.com/get-started/docker-overview/ (visited on 05/23/2026).
[52] docker.com. *What is a Container?* URL: https://www.docker.com/resources/what-container/ (visited on 05/23/2026).
[53] Axios. *Axios Documentation: Getting Started*. URL: https://axios-http.com/docs/intro (visited on 05/23/2026).
[54] Daniela Sele and Marina Chugunova. "Putting a human in the loop: Increasing uptake, but decreasing accuracy of automated decision-making". In: *PLOS ONE* 19.2 (2024), e0298037. DOI: 10.1371/journal.pone.0298037. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC10857587/ (visited on 06/03/2026).
[55] The Open University. *Approaches to Software Development: The Role of Development Processes*. URL: https://www.open.edu/openlearn/science-maths-technology/approaches-software-development/content-section-2.2 (visited on 06/02/2026).
[56] Christoph Becker et al. "Sustainability Design and Software: The Karlskrona Manifesto". In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 2. IEEE. 2015, pp. 467–476. URL: https://www.cs.toronto.edu/~sme/papers/2015/Beckeretal-ICSE2015.pdf (visited on 06/03/2026).
[57] United Nations. *The 17 Sustainable Development Goals*. URL: https://sdgs.un.org/goals (visited on 06/01/2026).

APPENDICES

A - GITHUB REPOSITORIES

All code and LaTeX files used in this document are included in the GitHub organization linked below. Further explanations are given in the README file.

- https://github.com/orgs/NTNU-Bachelor-2026-AIS/repositories

B - FORPROSJEKTSPLANEN

The pre-project plan (Forprosjektsplanen) submitted at the start of the project is included below.

Prosjektnr 2

Visualisering av anomalideteksjon av AIS-meldinger
Forprosjektplan

Versjon <1.0>

Prosjektnr 2

Revisjonshistorie

Dato	Versjon	Beskrivelse	Forfatter
<dd/mmm/yy>	<x.x>	<detaljer>	<navn>
9/1/2026	<0.5>	første versjon	aleksander nesvik, nikolai aambø, daniel alnes
1/19/2026		Ferdiggjøre risk analyse og diverse.	aleksander nesvik, daniel alnes
28/01/2026	1.0	Feriddgjøre timeplan og dokumentet	aleksander nesvik, daniel alnes

Prosjektnr 2

Innholdsfortegnelse

Prosjektnr 2

1. Mål og rammer

1.1 Orientering

Denne oppgaven var førstevalget gruppen hadde av 5 valg, gruppen valgte oppgaven fordi den virket interessant, og å jobbe innenfor maritim sikkerhet med kystverket er spennende.

1.2 Problemstilling / prosjektbeskrivelse og resultatmål

Gruppen skal lage visualiseringssoftware av skips-anomalier i norsk farvann, skip sender periodisk ut meldinger om deres posisjon på havet, og det dukker opp anomalier der skip kan teleportere, gi feil melding med vilje eller utstyrsfeil på skip som gjør at det kystverket må undersøke, gruppen skal lage ei side der de tar inn data fra anomalier og skal lage en bedre side enn nåværende løsning kystverket har for visualisering av slike anomalier. gruppen var gitt en mer fleksibel oppgave, slik at hva som skal visualiseres kan velges mer fritt, om det vil bygges ut tilleggsfunksjonalitet. målet er at gruppen har laget enten en prototype eller fullverdig ny visualiseringsapplikasjon som kystverket kan bygge ut eller ta i bruk selv.

1.3 Effektmål

Kystverket vil ha et bedre visualiseringsverktøy for anomalier, om dette blir oppnådd får de da gevinst i et nytt verktøy de kan bruke som kan utvides og bli brukt av kystverket for å lettere se anomalier og ta tak basert på lettliggjort informasjon.

1.4 Rammer

Ingen spesifikke behov for penger, men data fra kystverket og tilgang til noen av deres systemer trengs for oppgaven, samt informasjon. Ingen spesialbehov for materialer eller rom.

2. Organisering

Kystverket.

3. Gjennomføring

3.1. Hovedaktiviteter

Hovedaktiviteter er: rapportskriving, møtereferatskriving, innkalling til møter, koding av prosjektet.

Dokumentansvarlig er ansvarlig for møtereferat, andre forskjellige dokumenter er gruppen selv ansvarlig for, der dokumentansvarlig er den som er ansvarlig for at alt blir gjort. Gruppeleder er ansvarlig for innkalling til møter og andre administrative roller. Alle på gruppen er ansvarlig for føring av egne timer og arbeide, samt skal alle på gruppen jobbe med programmering og på selve bachelorrapporten.

3.2. Milepæler

15. Februar. Opplastning av forprosjektsplan.

18. mars. muntlig presentasjon om prosjektet på engelsk

14. mai. poster ferdiggjort og levert.

20. mai. Endelig innlevering av prosjektet.

4. Oppfølging og kvalitetssikring

4.1 Kvalitetssikring

Med å ha møter ofte der man går over arbeidet gjort og deretter kvalitetssikrer for å holde høy kvalitet. Å ha god kommunikasjon innen gruppen om noe går galt i prosjektet, om man sitter fast eller ikke klarer en oppgave etc. Gruppemedlemmer skal også ikke arbeide på oppgaver som ikke er tildelt uten å først kommunisere og få dette godkjent med resten av gruppen.

4.2 Rapportering

Minimum skal det bli rapportert til veileder og kunde en gang hver andre uke, dette er tenkt som et godt intervall slik at arbeid kan bli gjort og vist fram.

5. Risikovurdering

Risikoanalyse som vurderer sårbarheter i prosjektet (hendelse, sannsynlighet, konsekvens og tiltak).

KI var brukt for å finne forskjellige hendelser som vi ikke tenkte om og som hjelp for å finne løsninger

Svært sannsynlig	En dag passerer uten arbeid			Flere dager passerer uten arbeid	
Sannsynlig			Tidsbruk på vanskelig oppgave	Tidsklemme mot innlevering	
Mulig		Veileder er utilgjengelig	Manglende tilbakemelding fra oppdragsgiver	Datatap	
Lite sannsynlig		Sykdom i gruppen			Datalekkasje
Usannsynlig					Brudd på NDA
	Ubetydelig	Lav	Middels	Alvorlig	Kritisk

Prosjektnr 2

Detaljert Risikoanalyse

Hendelse	Tiltak og Beredskap
En/flere dager uten fremdrift	**Tiltak:** Daily Stand-up (daglige morgenmøter) forplikter alle til å si hva de skal gjøre i dag. Dette skaper sosial press og synlighet. **Beredskap:** Hvis Burndown Chart viser null fremdrift midt i en sprint, kaller vi inn til et møte for å re-planlegge sprinten.
Tidsbruk på vanskelig oppgave	**Tiltak:** Oppgaver estimeres under Sprint planlegging for å avdekke kompleksitet tidlig. Vanskelige oppgaver brytes ned i mindre deloppgaver. **Beredskap:** Vi tar opp hindringer i Daily Stand-up slik at Scrum Master eller andre i teamet kan hjelpe umiddelbart, i stedet for å vente til sprinten er over.
Tidsklemme mot innlevering	**Tiltak:** Vi jobber i korte Sprints for å ha kontinuerlig leveranse. Vi bruker Product Backlog for å prioritere de viktigste oppgavene (User Stories) først. **Beredskap:** Hvis vi ser i Sprint Review at vi henger etter, flytter vi mindre viktige oppgaver nederst i backloggen og fokuserer kun på MVP (Minimum Viable Product).
Veileder er utilgjengelig	**Tiltak:** Avtale faste møter. Samle opp spørsmål til e-poster fremfor mange små henvendelser. **Beredskap:** Søke svar i litteratur, hos andre faglærere, eller ta egne begrunnede valg og dokumentere disse i rapporten.
Manglende tilbakemelding fra oppdragsgiver	**Tiltak:** Avtale faste statusmøter. Tydelig kommunisere frister for når dere trenger svar. **Beredskap:** Gruppen tar egne avgjørelser basert på faglig forståelse dersom svar uteblir, og informerer oppdragsgiver om dette ("Vi antar X hvis vi ikke hører noe innen dato Y").
Datatap	**Tiltak:** Bruk av versjonskontroll (Git) og automatisk skylagring (OneDrive/Google Drive). **Beredskap:** Siden alt ligger i skyen/repo, kan arbeidet gjenopptas fra en annen PC umiddelbart.
Sykdom i gruppen	**Tiltak:** Godt arbeidsmiljø og jevnlige pauser. **Beredskap:** Parprogrammering og god dokumentasjon sikrer at gjenværende gruppemedlemmer kan fortsette arbeidet selv om en er borte.
Datalekkasje	**Tiltak:** Ingen sensitive data (API-nøkler, passord, kundedata) i kildekoden. Bruk av ‘ .gitignore’ og bruk av testdata. **Beredskap:** Varsle oppdragsgiver og veileder umiddelbart. Stenge

	tilganger og bytte passord/nøkler. Slette lekket data hvis mulig.
Brudd på NDA	**Tiltak:** Alle gruppemedlemmer signerer NDA. Oppgaven skrives generalisert der det er nødvendig. Data anonymiseres før det brukes i rapporten. **Beredskap:** Kontakt med bedriften for å begrense skade.

6. Vedlegg

Følgende dokumenter leveres som separate filer ved innlevering i Blackboard i januar (obligatorisk arbeidskrav), men iskke i endelige leveransen av hovedrapporten den 20. mai!

6.1 Tidsplan

Beskriv overordnet plan for faser i prosjektet. Det kan være, f.eks. Gantt-diagram eller eksport fra overordnet plan i Confluence/Wiki. Faser og/eller hovedaktiviteter må settes i tidsperspektiv – hva skjer i hvilke uker?

1. Forberedelse og planlegging	Avklare problemstilling, mål og krav. Jobb på dokumenter. Sett opp Git repository.	Uke 1–2
2. Design og forprosjekt.	Teknologivalg, skisse wireframe, prøve å ferdiggjøre dokumenter. Sett opp github issues og timeline for sprint etc.	Uke 3–5
3. Oppsett av nettside	Sette opp siden for prosjektet, dokumenter og mockup av siden er ferdiggjort og levert.	Uke 6
4. Integrere data, funksjonalitet for siden	Implementere funksjonalitet for siden som kunden vil ha, integrere dummy data vi har fått tilgang til.	Uke 7–15
5. Ferdiggjøring av prosjektet	Ferdiggjøring av bachelor rapport, og prosjektet.	Uke

og rapporten		15-20
6. Ferdigstilling og levering	Korrektur, ferdigstilling av rapport og prosjekt, innlevering. presentasjon av prosjektet.	Uke 21

6.2 Adresseliste

Navn, firma, tlf., epost, adresse

Marius Kalvø, Kystverket, marius.kalvo@kystverket.no, +47 46796934

6.3 Avtaledokumenter

6.3.1 Arbeidskontrakt for bachelor-gruppen

Arbeidskontrakt for 2026 bachelorgruppe 8

Medlemmer: Aleksander Lid Nesvik, Daniel Skjong Alnes, Nikolai Mork Aambø

Innledende tekst

Denne arbeidskontrakten bygger på et sett med typiske mål, oppgavefordelinger, prosedyrer og retningslinjer for interaksjoner for studentarbeider. Arbeidskontrakten er utfylt med *egne* fortolkninger av hva man mener med disse og hvordan man skal oppnå dette.

Roller og oppgavefordeling *(Hvordan organiserer man arbeidet?)*

Lagleder - <Daniel>

Den som leder gruppen. Ikke en sjef som bestemmer for hele gruppen. Lagleder er, hvis ingen andre er frivillig, møteleder og den som tar kontakt utenfor gruppen. Utenfor gruppen som for eksempel veileder eller kontaktperson hos oppdragsgiver.

Dokumentansvarlig - <Aleksander Nesvik>

Hovedsakelig møtereferent, men har ansvar over dokumenter generelt.

Prosjektnr 2

Kvalitetssikring - <Nikolai>

Den som sikrer standarder, skikk og orden i prosjektet og ikke blant gruppemedlemmer. Hovedansvar for riktighet i leveransen. Har også ansvar for god struktur i github repository.

Prosedyrer *(hvordan gjør man ting?)*

A. *Møteinnkalling*

Gruppen møter internt 1 gang i uken på tirsdag innimellom oppsatte INGA2301 timer. Gruppen møter også veileder og kontaktpersonen fra Kystverket, hver for seg, 1 gang annenhver uke. Møtestedet og tidspunktet arrangeres i samarbeid med veileder eller kontaktpersonen. Utnevnte møteleder sender invitasjon til alle som skal delta. Invitasjonen skal inneholde sted, dato, deltagere og mulighet for digitalt oppmøte.

B. *Varsling ved fravær eller andre hendelser*

Fravær, forsentkomming eller potensiell forsentkomming skal meldes om så tidlig som mulig. Fravær skal helst meldes med tekst slik at beskjeden kan hentes igjen. Helst skal alle relevante personer bli varslet, men det er nok at kun en person blir varslet hvis den personen er til stede for planen.

C. *Dokumenthåndtering*

Git og Github vil bli brukt til versjonskontroll og lagring av dokumenter. Alle dokumenter oppbevares i en repository, dedikert til administrative filer, på github. Samskriving blir håndtert av kommunikasjon og planlegging.

D. *Innleveringer av gruppearbeider*

En punktliste med hva som skal være i stand til innleveringen skal være laget på forhånd. Denne punktlisten skal bli brukt for å sørge for en fullstendig innlevering. Hvert repository på github skal ha en 'branch' som inneholder nyeste, leverbare, versjon av repository sitt innhold. Discord har en funksjon for servere som varsler medlemmer om kommende hendelser. Denne 'Event'-funksjonen skal brukes for å hjelpe alle medlemmene til å holde viktige frister. Gruppen skal ha ukentlige sprints med scrum, slik at frister skal kunne holdes med jevnt arbeid. Kvalitetskontroll vil bli gjort av alle gruppemedlemmer, med spisset fokus fra medlemmet med kvalitetssikring-rollen.

E. *Utviklingsmetode*

Gruppen skal bruke scrum med 1 uke lang sprinter. Scrum-møtet skal holdes på tirsdag

innimellom oppsatte INGA2301 timer. Alle medlemmer skal møte fysisk eller digitalt til møtet slik at alle er bevisste på, og kan diskutere i fellesskap om hva arbeid som gjenstår og må gjøres i neste sprint.

Interaksjon *(Hvordan opptrer man sammen?)*

A. ***Oppmøte og forberedelse***

Oppmøtetidspunkt blir satt av gruppe selv før møtet, som blir godtatt av hele gruppen før. Krav til forberedelser er å ha en god forståelse av hvor gruppen ligger an, om mer forberedelser trengs så blir dette kommunisert

B. ***Tilstedeværelse og engasjement***

Det er lov med pauser, og underholdning så lenge det ikke går ut over arbeidseffektiviteten. Gruppen bør holde seg engasjert slik at medlemmen ikke blir brent ut og demotivert.

C. ***Hvordan støtte hverandre***

At gruppen har fått god framgang og at medlemmene holder seg positive, at man ikke jobber for mye sent inn i prosjektet og at det har blitt gjort arbeid fra dag en gjør at gruppen kan glede seg til neste arbeidsdag.

D. ***Uenighet, avtalebrudd***

Gruppen aksepterer avvik som at ikke alle kan møte opp hver dag etc, men det blir mer problemer om dette ikke er kommunisert. Uenighet skal først prøves å løse via kommunikasjon innad i gruppen, om dette ikke fungerer så kan det bli møte med veileder eller emneansvarlig.

6.3.2 3-partsavtale

NTNU

Norges teknisk-naturvitenskapelige universitet

Fastsatt av prorektor for utdanning 10.12.2020

STANDARDAVTALE

om utføring av studentoppgave i samarbeid med ekstern virksomhet

Avtalen er ufravikelig for studentoppgaver (heretter oppgave) ved NTNU som utføres i samarbeid med ekstern virksomhet.

Forklaring av begrep

Opphavsrett
Er den rett som den som skaper et åndsverk har til å fremstille eksemplar av åndsverket og gjøre det tilgjengelig for allmennheten. Et åndsverk kan være et litterært, vitenskapelig eller kunstnerisk verk. En studentoppgave vil være et åndsverk.

Eiendomsrett til resultater
Betyr at den som eier resultatene bestemmer over disse. Utgangspunktet er at studenten eier resultatene fra sitt studentarbeid. Studenten kan også overføre eiendomsretten til den eksterne virksomheten.

Bruksrett til resultater
Den som eier resultatene kan gi andre en rett til å bruke resultatene, f.eks. at studenten gir NTNU og den eksterne virksomheten rett til å bruke resultatene fra studentoppgaven i deres virksomhet.

Prosjektbakgrunn
Det partene i avtalen har med seg inn i prosjektet, dvs. som vedkommende eier eller har rettigheter til fra før og som brukes i det videre arbeidet med studentoppgaven. Dette kan også være materiale som tredjepersoner (som ikke er part i avtalen) har rettigheter til.

Utsatt offentliggjøring
Betyr at oppgaven ikke blir tilgjengelig for allmennheten før etter en viss tid, f.eks. før etter tre år. Da vil det kun være veileder ved NTNU, sensorene og den eksterne virksomheten som har tilgang til studentarbeidet de tre første årene etter at studentarbeidet er innlevert.

NTNU

Norges teknisk-naturvitenskapelige universitet

Fastsatt av prorektor for utdanning 10.12.2020

STANDARDMAL ved avtale om konfidensialitet mellom student og ekstern virksomhet i forbindelse med studentens utførelse av oppgave (master-, bachelor- eller annen oppgave) i samarbeid med ekstern virksomhet, jf. punkt 9 i standardavtale om utføring av oppgave i samarbeid med ekstern virksomhet.

Student ved NTNU: Daniel Skjerve Klnes Fødselsdato: 16/11/2003
Hvis flere studenter: Alexander Lid Nesvik 26.02.1998 Nikolai Mork Aamb 21.06.2003
Ekstern virksomhet: Kystverket

1. Studenten skal utføre oppgave i samarbeid med ekstern virksomhet som ledd i sitt studium ved NTNU.

2. Studenten forplikter seg til å bevare taushet om det han/hun får vite om tekniske innretninger og fremgangsmåter samt drifts- og forretningsforhold som det vil være av konkurransemessig betydning å hemmeligholde for den eksterne virksomheten. Det er den eksterne sitt ansvar å sørge for å synliggjøre og tydeliggjøre hvilken informasjon dette omfatter.

3. Studenten er forpliktet til å bevare taushet om dette i 5 år regnet fra sluttdato.

4. Kravet om konfidensialitet gjelder ikke informasjon som:

a) var allment tilgjengelig da den ble mottatt
b) ble mottatt lovlig fra tredjeperson uten avtale om taushetsplikt
c) ble utviklet av studenten uavhengig av mottatt informasjon
d) partene er forpliktet til å gi opplysninger om i samsvar med lov eller forskrift eller etter pålegg fra offentlig myndighet.

Signaturer

Student: Dato: 14/01-2026 Hvis flere studenter: Daniel Klnes Nikolai M. Aamb Alexander Lid Nesvik
Ekstern virksomhet: Dato: 09/01-2026

1. Avtaleparter

Norges teknisk-naturvitenskapelige universitet (NTNU) Institutt: IKT og Realfag
Veileder ved NTNU: e-post og tlf. Di Wu di.wu@ntnu.no . 90947513
Ekstern virksomhet: kystverket Ekstern virksomhet sin kontaktperson, e-post og tlf.: Marius Kalvø, marius.kalvo@kystverket.no 46796934
Student: Daniel Skjong Almes Fødselsdato: 16/11/2003
Ev. flere studenter[1] Alexander Lid Nesvik 26.02.1998 Nikolai Mork Aambø 21.06.2003

Partene har ansvar for å klarere eventuelle immaterielle rettigheter som studenten, NTNU, den eksterne eller tredjeperson (som ikke er part i avtalen) har til prosjektbakgrunn før bruk i forbindelse med utførelse av oppgaven. Eierskap til prosjektbakgrunn skal fremgå av eget vedlegg til avtalen der dette kan ha betydning for utførelse av oppgaven.

2. Utførelse av oppgave

Studenten skal utføre: (sett kryss)

Masteroppgave	
Bacheloroppgave	X
Prosjektoppgave	
Annen oppgave	

Startdato:
Sluttdato:

Oppgavens arbeidstittel er: Visualisering av anomalideteksjon av AIS-meldinger

[1] Dersom flere studenter skriver oppgave i fellesskap, kan alle føres opp her. Rettigheter ligger da i fellesskap mellom studentene. Dersom ekstern virksomhet i stedet ønsker at det skal inngås egen avtale med hver enkelt student, gjøres dette.

Ansvarlig veileder ved NTNU har det overordnede faglige ansvaret for utforming og godkjenning av prosjektbeskrivelse og studentens læring.

3. Ekstern virksomhet sine plikter

Ekstern virksomhet skal stille med en kontaktperson som har nødvendig faglig kompetanse til å gi studenten tilstrekkelig veiledning i samarbeid med veileder ved NTNU. Ekstern kontaktperson fremgår i punkt 1.

Formålet med oppgaven er studentarbeid. Oppgaven utføres som ledd i studiet. Studenten skal ikke motta lønn eller lignende godtgjørelse fra den eksterne for studentarbeidet. Utgifter knyttet til gjennomføring av oppgaven skal dekkes av den eksterne. Aktuelle utgifter kan for eksempel være reiser, materialer for bygging av prototyp, innkjøp av prøver, tester på lab, kjemikalier. Studenten skal klarere dekning av utgifter med ekstern virksomhet på forhånd.

Ekstern virksomhet skal dekke følgende utgifter til utførelse av oppgaven:

Dekning av utgifter til annet enn det som er oppført her avgjøres av den eksterne underveis i arbeidet.

4. Studentens rettigheter

Studenten har opphavsrett til oppgaven[2]. Alle resultater av oppgaven, skapt av studenten alene gjennom arbeidet med oppgaven, eies av studenten med de begrensninger som følger av punkt 5, 6 og 7 nedenfor. Eiendomsretten til resultatene overføres til ekstern virksomhet hvis punkt 5 b er avkrysset eller for tilfelle som i punkt 6 (overføring ved patenterbare oppfinnelser).

I henhold til lov om opphavsrett til åndsverk beholder alltid studenten de ideelle rettigheter til eget åndsverk, dvs. retten til navngivelse og vern mot krenkende bruk.

Studenten har rett til å inngå egen avtale med NTNU om publisering av sin oppgave i NTNUs institusjonelle arkiv på Internett (NTNU Open). Studenten har også rett til å publisere oppgaven eller deler av den i andre sammenhenger dersom det ikke i denne avtalen er avtalt begrensninger i adgangen til å publisere, jf. punkt 8.

5. Den eksterne virksomheten sine rettigheter

Der oppgaven bygger på, eller videreutvikler materiale og/eller metoder (prosjektbakgrunn) som eies av den eksterne, eies prosjektbakgrunnen fortsatt av den eksterne. Hvis studenten

[2] Jf. Lov om opphavsrett til åndsverk mv. av 15.06.2018 § 1

skal utnytte resultater som inkluderer den eksterne sin prosjektbakgrunn, forutsetter dette at det er inngått egen avtale om dette mellom studenten og den eksterne virksomheten.

Alternativ a) (sett kryss) Hovedregel

X	Ekstern virksomhet skal ha bruksrett til resultatene av oppgaven

Dette innebærer at ekstern virksomhet skal ha rett til å benytte resultatene av oppgaven i egen virksomhet. Retten er ikke-eksklusiv.

Alternativ b) (sett kryss) Unntak

	Ekstern virksomhet skal ha eiendomsretten til resultatene av oppgaven og studentens bidrag i ekstern virksomhet sitt prosjekt

Begrunnelse for at ekstern virksomhet har behov for å få overført eiendomsrett til resultatene:

6. Godtgjøring ved patenterbare oppfinnelser

Dersom studenten i forbindelse med utførelsen av oppgaven har nådd frem til en patenterbar oppfinnelse, enten alene eller sammen med andre, kan den eksterne kreve retten til oppfinnelsen overført til seg. Dette forutsetter at utnyttelsen av oppfinnelsen faller inn under den eksterne sitt virksomhetsområde. I så fall har studenten krav på rimelig godtgjøring. Godtgjøringen skal fastsettes i samsvar med arbeidstakeroppfinnelsesloven § 7. Fristbestemmelsene i § 7 gis tilsvarende anvendelse.

7. NTNU sine rettigheter

De innleverte filer av oppgaven med vedlegg, som er nødvendig for sensur og arkivering ved NTNU, tilhører NTNU. NTNU får en vederlagsfri bruksrett til resultatene av oppgaven, inkludert vedlegg til denne, og kan benytte dette til undervisnings- og forskningsformål med de eventuelle begrensninger som fremgår i punkt 8.

8. Utsatt offentliggjøring

Hovedregelen er at studentoppgaver skal være offentlige.

Sett kryss

X	Oppgaven skal være offentlig

I særlige tilfeller kan partene bli enige om at hele eller deler av oppgaven skal være undergitt utsatt offentliggjøring i maksimalt tre år. Hvis oppgaven unntas fra offentliggjøring, vil den kun være tilgjengelig for student, ekstern virksomhet og veileder i denne perioden. Sensurkomiteen vil ha tilgang til oppgaven i forbindelse med sensur. Student, veileder og sensorer har taushetsplikt om innhold som er unntatt offentliggjøring.

Oppgaven skal være underlagt utsatt offentliggjøring i (sett kryss hvis dette er aktuelt):

Sett kryss		Sett dato
	ett år	
	to år	
	tre år	

Behovet for utsatt offentliggjøring er begrunnet ut fra følgende:

Dersom partene, etter at oppgaven er ferdig, blir enig om at det ikke er behov for utsatt offentliggjøring, kan dette endres. I så fall skal dette avtales skriftlig.

Vedlegg til oppgaven kan unntas ut over tre år etter forespørsel fra ekstern virksomhet. NTNU (ved instituttet) og student skal godta dette hvis den eksterne har saklig grunn for å be om at et eller flere vedlegg unntas. Ekstern virksomhet må sende forespørsel før oppgaven leveres.

De delene av oppgaven som ikke er undergitt utsatt offentliggjøring, kan publiseres i NTNUs institusjonelle arkiv, jf. punkt 4, siste avsnitt. Selv om oppgaven er undergitt utsatt offentliggjøring, skal ekstern virksomhet legge til rette for at studenten kan benytte hele eller deler av oppgaven i forbindelse med jobbsøknader samt videreføring i et master- eller doktorgradsarbeid.

9. Generelt

Denne avtalen skal ha gyldighet foran andre avtaler som er eller blir opprettet mellom to av partene som er nevnt ovenfor. Dersom student og ekstern virksomhet skal inngå avtale om konfidensialitet om det som studenten får kjennskap til i eller gjennom den eksterne virksomheten, kan NTNUs standardmal for konfidensialitetsavtale benyttes.

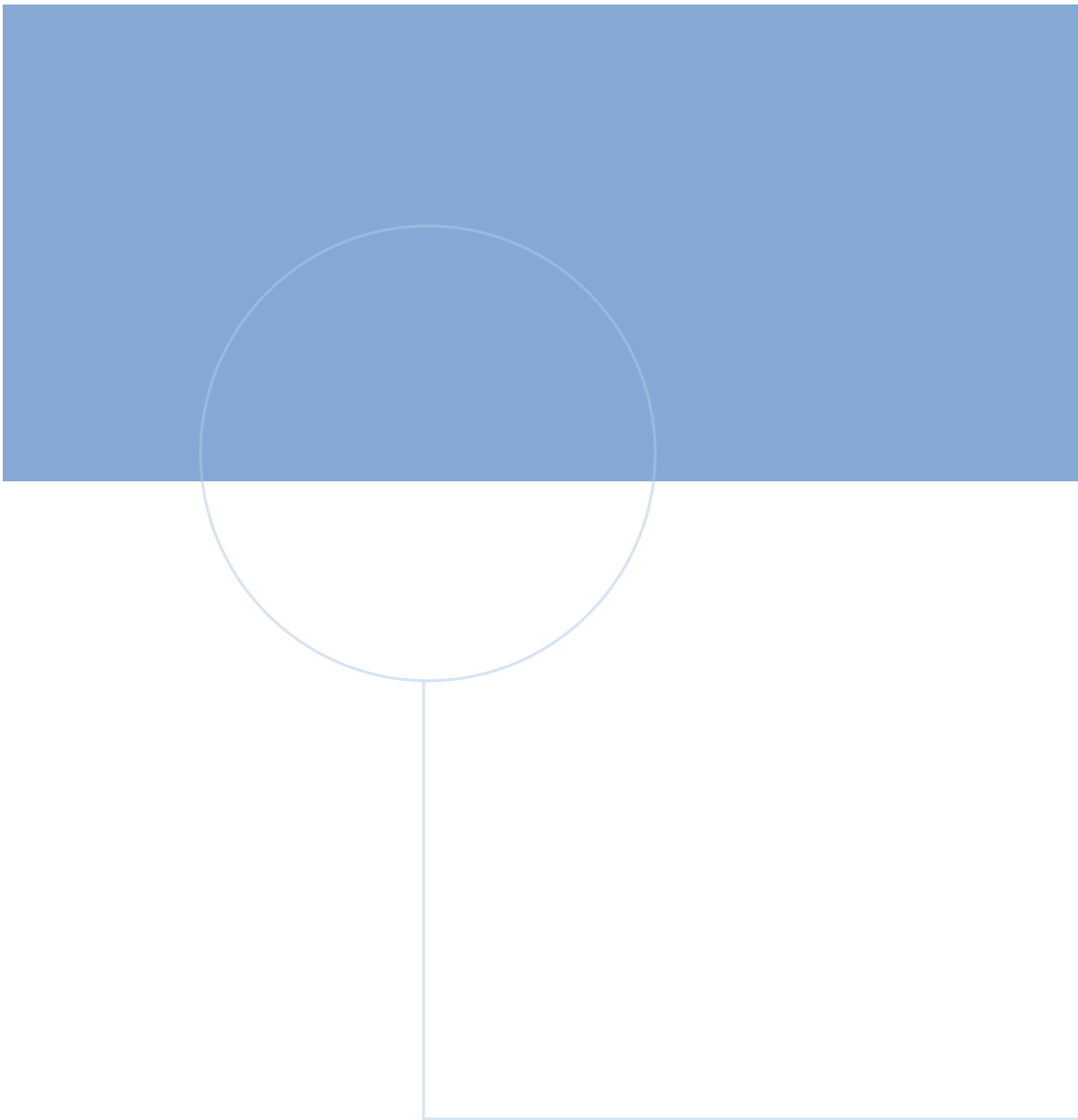
Den eksterne sin egen konfidensialitetsavtale, eventuell konfidensialitetsavtale den eksterne har inngått i samarbeidprosjekter, kan også brukes forutsatt at den ikke inneholder punkter i motstrid med denne avtalen (om rettigheter, offentliggjøring mm). Dersom det likevel viser seg at det er motstrid, skal NTNUs standardavtale om utføring av studentoppgave gå foran. Eventuell avtale om konfidensialitet skal vedlegges denne avtalen.

Eventuell uenighet som følge av denne avtalen skal søkes løst ved forhandlinger. Hvis dette ikke fører frem, er partene enige om at tvisten avgjøres ved voldgift i henhold til norsk lov. Tvisten avgjøres av sorenskriveren ved Sør-Trøndelag tingrett eller den han/hun oppnevner.

Denne avtale er signert i fire eksemplarer hvor partene skal ha hvert sitt eksemplar. Avtalen er gyldig når den er underskrevet av NTNU v/instituttleder.

Signaturer:

Instituttleder: Dato:
Veileder ved NTNU: Dato: 15/01-26
Ekstern virksomhet: Kystverket v/ Dato: 14/1-26
Student: Dato: 15/01-26 Daniel K Aleksander N. Nikolai A.
Ev. flere studenter



NTNU

Norwegian University of
Science and Technology